UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

JULIANA RODRIGUES DE VARGAS

# Hardware Accelerator for Matrix Inversion in Decentralized Massive MIMO LMMSE Equalization Under Colored Noise

Work presented in partial fulfillment
of the requirements for the degree of
Bachelor in Computer Engineering

Advisor: Prof. Dr. Leslie Lamport
Coadvisor: Prof. Dr. Donald Ervin Knuth

Porto Alegre
November 2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Reitor: Prof. Rui Vicente Oppermann
Vice-Reitora: Prof[a]. Jane Fraga Tutikian
Pró-Reitor de Graduação: Prof. Vladimir Pinheiro do Nascimento
Diretora do Instituto de Informática: Prof[a]. Carla Maria Dal Sasso Freitas
Coordenador do Curso de Engenharia de Computação: Prof. André Inácio Reis
Bibliotecária-chefe do Instituto de Informática: Beatriz Regina Bastos Haro

**ABSTRACT**

Este documento é um exemplo de como formatar documentos para o Instituto de Informática da UFRGS usando as classes LaTeX disponibilizadas pelo UTUG. Ao mesmo tempo, pode servir de consulta para comandos mais genéricos. *O texto do resumo não deve conter mais do que 500 palavras.*

**Keywords:** Formatação eletrônica de documentos. LaTeX. ABNT. UFRGS.

**Using LaTeX to Prepare Documents at II/UFRGS**

## RESUMO

This document is an example on how to prepare documents at II/UFRGS using the LaTeX classes provided by the UTUG. At the same time, it may serve as a guide for general-purpose commands. *The text in the abstract should not contain more than 500 words.*

**Palavras-chave:** Electronic document preparation. LaTeX. ABNT. UFRGS.

## LIST OF FIGURES

Figure 2.1 Resource elements of one PRB in one slot, with the DM-RS symbols. ..... 25
Figure 2.2 Intervals of duration $T_c$ that contain a data symbol, and their DM-RS REs per PRB, in the TDLB100-400 condition. ..... 26
Figure 2.3 Lower triangular systolic array that computes a Gram matrix. ..... 32

Figure 3.1 Simulation flow: QuaDRiGa channels, Python simulator and fixed-point inverse. ..... 34
Figure 3.2 Detailed flow of the Python simulator: Monte Carlo loop, equalizers and SER. ..... 36
Figure 3.3 Steps of the inversion module. Each block is one C++ function of the HLS code. ..... 38
Figure 3.4 Loop pipelining in the LU decomposition. The code is simplified. ..... 48
Figure 3.5 Dataflow between the stages of the inversion. The code is simplified. ..... 49
Figure 3.6 Batch of 12 matrices with 6 copies, for the 606-cycle limit. The code is simplified. ..... 49
Figure 3.7 Systolic array of the forward substitution and minimum step of the LU decomposition. The code is simplified. ..... 51

## LIST OF TABLES


Table 2.1 Tempo de coerência de cada condição de teste. ..... 16
Table 2.2 Cenários do modelo 3GPP 38.901 no QuaDRiGa. ..... 17
Table 2.3 Number of DM-RS REs per PRB available to every data symbol of the slot, in each test condition. ..... 26
Table 2.4 Multiplicações complexas por PRB e por $T_c$ em cada DU. ..... 29
Table 2.5 Modos de quantização e de estouro do tipo `ap_fixed`. ..... 31

Table 3.1 Types of the values in the inversion module, for the format $\langle W, I \rangle$ with $F = W - I$ fractional bits. ..... 38
Table 3.2 Simulated configurations. ..... 40
Table 3.3 Carrier frequency and number of noise samples in each test condition. ..... 40
Table 3.4 Maximum interval between inversions in each test condition. ..... 42
Table 3.5 Sizes tested in the initial sweep. ..... 43
Table 3.6 $\mathbf{G}_c$ at the operating point of the initial-sweep configurations. ..... 44
Table 3.7 Configurations in which each format was chosen in the initial sweep. ..... 45
Table 3.8 Batch of the inversion module in each test condition. ..... 47

Table 4.1 Formatos escolhidos em cada grupo de configurações. ..... 56
Table 4.2 Throughput and delay of one block of $N_{\text{PRB}} = 275$ inversions, format $\langle 24, 12 \rangle$. ..... 57

Table A.1 Channel parameters set in the MATLAB script that runs QuaDRiGa. ..... 61
Table A.2 Parameters of the propagation scenario `3GPP_38.901_UMa_NLOS`: valid range and values used in the simulation. ..... 62
Table A.3 Signal and noise parameters. ..... 62
Table A.4 Receiver parameters: partitioning, covariance estimation and equalization. ..... 63
Table A.5 Evaluation parameters. ..... 63

## LIST OF ABBREVIATIONS AND ACRONYMS

| | |
|---|---|
| 3GPP | 3rd Generation Partnership Project |
| 5G | Quinta geração de redes móveis |
| AoA | Angle of Arrival |
| AoD | Angle of Departure |
| AWGN | Additive White Gaussian Noise |
| BCD | Block Coordinate Descent |
| BS | Base Station |
| DM-RS | Demodulation Reference Signal |
| DU | Distributed Unit |
| FR1 | Frequency Range 1 |
| FR2 | Frequency Range 2 |
| HST | High Speed Train |
| InF | Indoor Factory |
| InH | Indoor Hotspot |
| IoT | Interference over Thermal |
| ITU | International Telecommunication Union |
| LMMSE | Linear Minimum Mean Square Error |
| LoS | Line of Sight |
| MIMO | Multiple-Input Multiple-Output |
| MMSE | Minimum Mean Square Error |
| NLoS | Non-Line of Sight |
| NR | New Radio |
| O2I | Outdoor-to-Indoor |
| OFDM | Orthogonal Frequency Division Multiplexing |

| | |
|---|---|
| PRB | Physical Resource Block |
| PUSCH | Physical Uplink Shared Channel |
| QAM | Quadrature Amplitude Modulation |
| QPSK | Quadrature Phase Shift Keying |
| QuaDRiGa | Quasi-Deterministic Radio Channel Generator |
| RB | Resource Block |
| RE | Resource Element |
| RMa | Rural Macro-cell |
| SCS | Subcarrier Spacing |
| SER | Symbol Error Rate |
| SNR | Signal-to-Noise Ratio |
| TR | Technical Report |
| TS | Technical Specification |
| UE | User Equipment |
| ULA | Uniform Linear Array |
| UMa | Urban Macro-cell |
| UMi | Urban Micro-cell |
| UPA | Uniform Planar Array |
| ZF | Zero-Forcing |

**CONTENTS**


**1 INTRODUÇÃO ..... 11**
**1.1 Figuras e tabelas ..... 11**
**2 FUNDAMENTAÇÃO TEÓRICA ..... 12**
**2.1 Sistemas MIMO ..... 12**
2.1.1 Uplink ..... 12
2.1.2 Símbolos e constelação ..... 12
2.1.3 Relação sinal-ruído ..... 14
2.1.4 Modelo de canal ..... 14
2.1.4.1 Tipos de modelo de canal ..... 14
2.1.4.2 Multipath, desvanecimento... ..... 15
2.1.4.3 Bloco de Coerência ..... 15
2.1.4.4 Arranjo de canal ..... 16
2.1.4.5 Modelo 3GPP e Simulador QuaDRiGa ..... 16
2.1.4.6 Cenário de propagação ..... 17
2.1.4.7 Parâmetros do simulador ..... 18
2.1.5 Equalização ..... 18
2.1.5.1 Detector de máxima verossimilhança ..... 18
2.1.5.2 Zero-forcing ..... 18
2.1.5.3 Covariância ..... 18
2.1.5.4 Filtro de Wiener ..... 18
2.1.5.5 Equalizador LMMSE ..... 18
2.1.5.6 Detecção e taxa de erro de símbolo ..... 18
**2.2 Equalização descentralizada sob ruído colorido ..... 18**
2.2.1 Ruído ..... 18
2.2.1.1 Ruído branco (AWGN) ..... 19
2.2.1.2 Assinatura espacial e Interferência ..... 19
2.2.1.3 Matriz de covariância do ruído ..... 21
2.2.1.4 Interferência sobre ruído térmico (IoT) ..... 22
2.2.1.5 Estimação da matriz de covariância ..... 23
2.2.1.6 Availability of noise samples within the coherence time ..... 24
2.2.1.7 Condicionamento da estimativa ..... 27
2.2.2 Processamento descentralizado ..... 27
2.2.3 Equalizador BCD-MMSE ..... 28
2.2.3.1 Complexidade ..... 29
**2.3 Inversão de Matrizes ..... 30**
2.3.1 Inversao de Matrizes ..... 30
2.3.1.1 Decomposicao de Cholesky ..... 30
2.3.1.2 Representação em ponto fixo ..... 30
2.3.1.3 Latência, intervalo e vazão ..... 30
2.3.1.4 Pipeline de tarefas ..... 31
2.3.1.5 Arranjo sistólico ..... 32
**3 METHODOLOGY ..... 33**
**3.1 Simulation workflow ..... 33**
3.1.1 Channel generation ..... 34
3.1.2 Python simulator ..... 35
3.1.3 Fixed-point inverse ..... 35
**3.2 Hardware setup ..... 37**
3.2.1 Tools and target device ..... 37

3.2.2 Inversion module ..... 37
**3.3 Configurations and test conditions ..... 39**
3.3.1 Base station configurations ..... 39
3.3.2 Test conditions ..... 39
**3.4 Inversion cycle limit ..... 41**
3.4.1 Sample collection ..... 41
3.4.2 Throughput ..... 41
**3.5 Selection of the fixed-point format ..... 42**
3.5.1 SNR increase ..... 42
3.5.2 Initial sweep ..... 43
3.5.3 Configurations of the initial sweep ..... 43
3.5.4 Synthesis and selection of formats for validation ..... 44
3.5.5 Validation ..... 44
**3.6 Hardware architectures, test matrices and power ..... 46**
3.6.1 Hardware architectures ..... 46
3.6.1.1 Loop pipelining ..... 47
3.6.1.2 Dataflow between stages ..... 47
3.6.1.3 Matrix batches ..... 48
3.6.1.4 Systolic arrays ..... 50
3.6.2 Test matrices ..... 50
3.6.3 Power and energy ..... 51
**3.7 Organization of the results ..... 52**
**4 RESULTADOS ..... 53**
**4.1 Simulation setup ..... 53**
4.1.1 Channel Generation in QuaDRiGa ..... 53
4.1.2 Python Simulator ..... 54
4.1.2.1 Signal and Noise ..... 54
4.1.2.2 Receiver ..... 55
4.1.2.3 Evaluation ..... 55
**4.2 Formato de ponto fixo ..... 55**
4.2.1 Mapas de aumento de SNR ..... 55
4.2.2 Formatos escolhidos ..... 55
**4.3 Vazão e atraso ..... 56**
4.3.1 Vazão ..... 56
4.3.2 Atraso ..... 56
**5 CONCLUSIONS ..... 59**
**5.1 Future work ..... 59**
**REFERENCES ..... 60**
**APPENDIX A — SIMULATION PARAMETERS ..... 61**

# 1 INTRODUÇÃO

## 1.1 Figuras e tabelas

# 2 FUNDAMENTAÇÃO TEÓRICA

## 2.1 Sistemas MIMO

### 2.1.1 Uplink

Este trabalho considera o *uplink* de um sistema MIMO (*Multiple-Input Multiple-Output*) massivo, em que $K$ Equipamentos de Usuário (UEs, *User Equipments*) alvo, cada um com uma antena, transmitem dados para uma Estação Base (BS, *Base Station*) com $M$ antenas, sendo $M \gg K$ (ZHAO et al., 2025). Os UEs são atendidos ao mesmo tempo, nos mesmos recursos de tempo e frequência (ZHAO et al., 2025).

O sistema segue o 5G NR (*New Radio*), a tecnologia de acesso sem fio da quinta geração de redes móveis (5G) (LIN; LEE, 2021, p. 2). O NR é especificado pelo 3GPP (*3rd Generation Partnership Project*) em um conjunto de normas técnicas, como a TS 38.211, que define os canais físicos e a modulação (3GPP, 2025), e a TS 38.104, que define a transmissão e a recepção da BS (3GPP, 2020). O NR divide o espectro em duas faixas de frequência: FR1 (*Frequency Range* 1), de 410 MHz a 7125 MHz, e FR2, de 24250 MHz a 52600 MHz (3GPP, 2020, Tab. 5.1-1). No *uplink*, os dados dos UEs são transmitidos no canal físico compartilhado de *uplink* (PUSCH, *Physical Uplink Shared Channel*) (3GPP, 2025, Sec. 6.3.1).

### 2.1.2 Símbolos e constelação

Cada UE transmite símbolos de uma constelação de modulação, como a 16-QAM (*Quadrature Amplitude Modulation*) (ZHAO et al., 2025). A energia média por símbolo é $E_s$ (ZHAO et al., 2025).

No NR, esses símbolos são transmitidos com OFDM (*Orthogonal Frequency Division Multiplexing*). O espaçamento entre subportadoras (SCS, *Subcarrier Spacing*) depende da numerologia $\mu$, com $\Delta f = 2^{\mu} \cdot 15$ kHz (3GPP, 2025, Tab. 4.2-1). Por exemplo, $\mu = 1$ corresponde a 30 kHz, e $\mu = 3$ corresponde a 120 kHz.

No tempo, as transmissões são organizadas em quadros de 10 ms, e cada quadro tem dez subquadros de 1 ms (3GPP, 2025, Sec. 4.3.1). O slot é a divisão do subquadro usada na transmissão. Com prefixo cíclico normal, cada slot tem 14 símbolos OFDM, e cada subquadro tem $2^{\mu}$ slots (3GPP, 2025, Tab. 4.3.2-1). Assim, um slot dura 0,5 ms com

$\mu = 1$ e 0,125 ms com $\mu = 3$.

O UE só transmite o PUSCH quando a BS lhe concede recursos para isso. Essa concessão indica o slot e os recursos de frequência do PUSCH (LIN; LEE, 2021, p. 293). Assim, um slot só leva o PUSCH de um UE quando esse UE tem dados para enviar e foi escalonado. O *uplink* e o *downlink* também precisam dividir o espectro. Na duplexação por divisão no tempo (TDD, *Time Division Duplex*), os dois usam a mesma banda de frequência, em tempos diferentes (LIN; LEE, 2021, p. 104). Na duplexação por divisão na frequência (FDD, *Frequency Division Duplex*), os dois usam bandas diferentes ao mesmo tempo (LIN; LEE, 2021, p. 104). O TDD é cada vez mais comum nas bandas de frequência mais altas (LIN; LEE, 2021, p. 10). Com TDD, só os slots de *uplink* podem levar o PUSCH.

A grade de recursos combina subportadoras, na frequência, e símbolos OFDM, no tempo. Cada elemento dessa grade é um elemento de recurso (RE, *Resource Element*), que corresponde a uma subportadora em um símbolo OFDM (3GPP, 2025, Sec. 4.4.3). Um bloco de recursos (RB, *Resource Block*) tem 12 subportadoras consecutivas na frequência (3GPP, 2025, Sec. 4.4.4.1). Os blocos de recursos físicos (PRBs, *Physical Resource Blocks*) são os RBs usados dentro da parte da banda configurada para a transmissão (3GPP, 2025, Sec. 4.4.4.4).

O número de PRBs da transmissão é $N_{\text{PRB}}$. A parte da banda configurada pode ter de 1 a 275 RBs (LIN; LEE, 2021, p. 367). Esse limite vem da grade de recursos: a norma numera as suas subportadoras de 0 a 3299 (3GPP, 2025, Sec. 4.4.2). São, portanto, até 3300 subportadoras. Como cada RB tem 12 subportadoras, isso equivale a $3300/12 = 275$ RBs. As larguras de banda da norma ficam um pouco abaixo desse limite. O maior número de PRBs é 273 com SCS de 30 kHz e banda de 100 MHz, e 264 com SCS de 120 kHz e banda de 400 MHz (3GPP, 2020, Tabs. 5.3.2-1 e 5.3.2-2). Com 275 PRBs, as subportadoras ocupariam $3300 \times 30$ kHz $= 99$ MHz e $3300 \times 120$ kHz $= 396$ MHz, quase toda a banda. Como o receptor processa cada PRB, $N_{\text{PRB}} = 275$ é o pior caso do 5G NR (ZHAO et al., 2025).

~~Resource Element (RE) é a menor unidade básica de transmissão, correspondendo a uma subportadora no domínio da frequência e um símbolo OFDM no domínio do tempo. Slot é a unidade básica de tempo para o agendamento de transmissões na rede, representando um bloco temporal contínuo. No padrão 5G NR, cada PRB possui 12 subportadores consecutivas na frequência, e cada slot possui 14 símbolos.~~

### 2.1.3 Relação sinal-ruído

A relação sinal-ruído (SNR, *Signal-to-Noise Ratio*) compara a energia média por símbolo com a potência do ruído térmico em cada antena, $\sigma^2$ (Seção 2.2.1). Ela é definida como (ZHAO et al., 2025)

$$\mathrm{SNR} = 10 \log_{10} \left( \frac{E_s}{\sigma^2} \right). \tag{2.1}$$

### 2.1.4 Modelo de canal

A avaliação de um sistema de MIMO massivo precisa de um modelo que represente o canal com as principais características de um arranjo com muitas antenas. Esse modelo deve considerar a geometria do arranjo de antenas, a correlação espacial entre as respostas de canal de diferentes antenas e as características físicas do ambiente, como a localização e orientação das ~~Estações Base (BSs) e dos Equipamentos de Usuário (UEs.)~~ BSs e dos UEs.

#### *2.1.4.1 Tipos de modelo de canal*

Existem dois tipos principais de modelo de canal: determinísticos ou estocásticos. O modelo determinístico produz sempre o mesmo resultado sob as mesmas condições, pois depende de um ambiente específico com parâmetros fixos, como a localização dos transmissores, receptores, espalhadores e refletores de sinal. Exemplos desse modelo são o ray–tracing e medições reais de canal em campo. Nesse modelo há desvantagem para a reprodução de resultados, que exige o acesso aos dados de medição específicos do canal ou modelos 3D de prédios, que nem sempre estão disponíveis.

Modelos estocásticos são independentes de um ambiente em particular e generalizam o comportamento do canal a partir de propriedades estatísticas selecionadas. Eles podem ser divididos em três categorias:

- Baseados em correlação: Nos modelos baseados em correlação as respostas do canal seguem uma distribuição gaussiana com média zero, definidas por meio de matrizes de correlação. Um exemplo é o correlated Rayleigh fading model, que considera um desvanecimento plano em frequência, ou seja, que todos os componentes multi-path chegam à antena com o mesmo delay. Esse modelo é suficiente

para uma análise dentro de um bloco de coerência ~~(PRB)~~ (um PRB, na frequência, durante $T_c$, no tempo), em que o canal é considerado constante.

- Paramétricos: Esses modelos definem distribuições estatísticas para o número de espalhadores, atrasos, potências e ângulos de chegada e saída (AoA/AoD). Eles não consideram a geometria do ambiente, por isso não abrangem a análise da evolução temporal do canal que acontece com a movimentação dos ~~Equipamentos de Usuário~~ UEs.
- Baseados em geometria: Esses modelos equilibram as características dos modelos estocásticos e determinísticos. Eles definem uma distribuição da localização física dos espalhadores de sinal ao redor dos transmissores e receptores. Os caminhos de propagação do canal são modelados a partir da localização dos espalhadores de uma maneira quase-determinística (quasi-deterministic). Um modelo quase-determinístico define um processo regido por leis fixas mas com uma margem de aleatoriedade dada por ruído e/ou variabilidade.

#### *2.1.4.2 Multipath, desvanecimento...*

#### *2.1.4.3 Bloco de Coerência*

O tempo de coerência $T_c$ corresponde ao intervalo durante o qual o canal pode ser considerado aproximadamente constante. Seu valor é calculado a partir do deslocamento Doppler máximo $f_D$, pela expressão $T_c = 1/(8f_D)$ (TSE; VISWANATH, 2005). Durante cada bloco de coerência, o canal e a matriz de covariância do ruído são considerados constantes, permitindo reutilizar a matriz de equalização $\mathbf{W}$ ao longo desse intervalo (ZHAO et al., 2025).

As condições de teste seguem os cenários de propagação definidos no Anexo G da norma 3GPP TS 38.104 (3GPP, 2020). Para os cálculos, são adotados ~~espaçamentos entre subportadoras (SCS)~~ SCS de 30 kHz em FR1 e 120 kHz em FR2 (3GPP, 2020, Tabs. 5.3.5-1 e 5.3.5-2). Considerando os 14 símbolos OFDM por slot (3GPP, 2025, Tab. 4.3.2-1), a Tabela 2.1 apresenta o tempo de coerência e o número de símbolos OFDM completos contidos nesse intervalo para cada condição de teste. A classificação de mobilidade típica baseia-se nos cenários de uso do 5G descritos pela ITU (ITU-R, 2015, pp. 11, 14).

Table 2.1: Tempo de coerência de cada condição de teste.

| Condição de teste | Mobilidade típica | $f_D$ (Hz) | $v$ (km/h) | SCS (kHz) | $T_c$ (ms) | Símb. OFDM |
|---|---|---|---|---|---|---|
| TDLA30-5 (FR1) | Baixa: *hotspot*, *indoor* | 5 | 1,5 | 30 | 25,0000 | 700 |
| TDLA30-10 (FR1) | Baixa: *hotspot*, *indoor* | 10 | 3,1 | 30 | 12,5000 | 350 |
| TDLA30-75 (FR2) | Baixa: *hotspot*, *indoor* | 75 | 2,9 | 120 | 1,6667 | 186 |
| TDLA30-300 (FR2) | Média a alta: área ampla, carros | 300 | 11,6 | 120 | 0,4167 | 46 |
| TDLC300-100 (FR1) | Média a alta: área ampla, carros | 100 | 30,8 | 30 | 1,2500 | 35 |
| TDLB100-400 (FR1) | Média a alta: área ampla, carros | 400 | 123,3 | 30 | 0,3125 | 8 |
| HST350 (FR1) | Alta: trem de alta velocidade | 2334 | 350 | 30 | 0,0536 | 1 |
| HST500 (FR1) | Alta: trem de alta velocidade | 3334 | 500 | 30 | 0,0375 | 1 |

#### *2.1.4.4 Arranjo de canal*

A escolha dos arranjos de antena (ou seja, suas distribuições no espaço) dependem do caso de uso, da frequência dos portadores e da área de cobertura. Os arranjos podem ser organizados em geometrias lineares (Uniform Linear Array - ULA), planares (Uniform Planar Array - UPA) ou em geometrias cilíndricas. Modelos lineares e planares separam os UEs baseados na elevação ou posição azimutal, enquanto os cilíndricos conseguem trabalhar nos dois domínios ao mesmo tempo. Para cenários urbanos, o arranjo cilíndrico é mais adequado, pois permite a representação dos UEs em diferentes localizações e andares de prédios.

#### *2.1.4.5 Modelo 3GPP e Simulador QuaDRiGa*

Linha de Visada (Line of Sight - LoS) é o trajeto livre de obstáculos que permite a visão direta entre dois pontos. Não é realista representar os caminhos de propagação apenas como linha de visada. A 3GPP definiu para sistemas MIMO um modelo estocástico baseado em geometria que representa um misto de caminhos de propagação NLos e LoS. Esse modelo une um layout determinístico do sistema MIMO (ocalização das BS e UE, orientação das antenas, dependência angular da intensidade das ondas de rádio transmitidas ou recebidas, frequência de subportadoras) e um conjunto de parâmetros aleatórios de larga escala gerados por distribuições estatísticas obtidas a partir de medições de canal. O simulador de canal de código aberto QuaDRiGa (Quasi-Deterministic Radio Channel Generator) implementa o modelo 3GPP 3D MIMO, e é o simulador utilizado neste trabalho.

#### *2.1.4.6 Cenário de propagação*

Um cenário de propagação descreve um conjunto de parâmetros estatísticos para um tipo de ambiente. Dentre os modelos de cenário, o trabalho utiliza o 3GPP NR (~~TR 39.901~~ TR 38.901), [que modela os parâmetros para o New Radio, que é uma frequencia maior, pior caso]. Os ambientes do modelo ~~3G– 38.901~~ 3GPP TR 38.901 podem ter as seguintes características:

- ~~Uma~~ UMa (*Urban Macro-cell*): cidade densa, com antenas acima dos telhados.
- ~~Umi~~ UMi (*Urban Micro-cell*): cidade densa, com uma antena pequena abaixo dos telhados, no nível da rua.
- RMa (*Rural Macro-cell*): área rural, com células grandes.
- InH (~~*Indoor Office*~~ *Indoor Hotspot*): dentro de prédios, como escritórios.
- InF (*Indoor Factory*): dentro de galpões de fábrica.

Além disso, cada ambiente pode ter variações:

- LOS/NLOS: com ou sem visada direta entre antena e usuário.
- O2I (*Outdoor-to-Indoor*): antena do lado de fora e usuário dentro de prédio.

Cada cenário do modelo é definido por um arquivo de configuração. A tabela 2.2 resume os cenários do modelo. Este trabalho usa o cenário `3GPP_38.901_UMa_NLOS`, que representa uma macrocélula urbana (*Urban Macro* - UMa) sem linha de visada (NLos) entre a BS e os UEs.

Table 2.2: Cenários do modelo 3GPP 38.901 no QuaDRiGa.

| Cenário | Lugar | Dados do manual |
|---|---|---|
| `3GPP_38.901_UMa` | Cidade densa, antena no alto do prédio | Célula de até ∼1 km, BS a 25 m, usuário de 1,5 a 22,5 m |
| `3GPP_38.901_UMi` | Cidade densa, antena na rua, abaixo dos telhados | Célula de até ∼200 m, BS a 10 m |
| `3GPP_38.901_RMa` | Área rural | Célula de até ∼10 km, BS de 10 a 150 m |
| `3GPP_38.901_Indoor` | Escritório, como Wi-Fi ou femtocélula | BS a 3 m, usuário a 1 m. Tem dois tipos, *mixed office* e *open office*, que só mudam a probabilidade de LOS. |
| `3GPP_38.901_InF` | Galpão de fábrica com máquinas, linhas de montagem e prateleiras | Versões NLOS por densidade de obstáculos (D: denso, S: esparso) e altura da BS (H: alta, L: baixa): DH, DL, SH e SL |

Fonte: manual do QuaDRiGa.

*2.1.4.7 Parâmetros do simulador*

Para além dos parâmetros fixos pelo cenário de simulação, uma série de parâmetros deve ser definida. Este trabalho utiliza os parâmetros definidos em (ZHAO et al., 2025).

### 2.1.5 Equalização

*2.1.5.1 Detector de máxima verossimilhança*

*2.1.5.2 Zero-forcing*

*2.1.5.3 Covariância*

*2.1.5.4 Filtro de Wiener*

*2.1.5.5 Equalizador LMMSE*

*2.1.5.6 Detecção e taxa de erro de símbolo*

Depois da equalização, cada entrada do vetor estimado $\hat{s}$ é levada ao ponto mais próximo da constelação (ZHAO et al., 2025). Essa é a regra do vizinho mais próximo (TSE; VISWANATH, 2005, p. 503). A detecção erra quando o ponto escolhido é diferente do símbolo transmitido. A taxa de erro de símbolo (SER, *Symbol Error Rate*) é a probabilidade desse erro (TSE; VISWANATH, 2005, p. 503). Ela cai quando a SNR aumenta, e o desempenho de um equalizador é mostrado pela curva de SER em função da SNR (ZHAO et al., 2025). Para comparar dois esquemas, é comum usar a diferença de SNR necessária para atingir a mesma probabilidade de erro, que é a distância horizontal entre as duas curvas (TSE; VISWANATH, 2005, p. 55).

## 2.2 Equalização descentralizada sob ruído colorido

### 2.2.1 Ruído

O equalizador LMMSE (*Linear Minimum Mean Square Error*) (ZHAO et al., 2025) [] depende da matriz de covariância do ruído, que descreve suas características estatísticas e a relação entre os sinais observados nas diferentes antenas da ~~estação~~

~~base~~ BS. Quando o ruído é apenas térmico (AWGN) e por consequência, espacialmente branco, essa matriz é proporcional à identidade. Entretanto, a presença da interferência de usuários não-alvo adiciona correlação espacial do ruído entre as antenas. Esta seção apresenta o modelo de ruído considerado neste trabalho e deriva sua matriz de covariância.

#### *2.2.1.1 Ruído branco (AWGN)*

No modelo AWGN (Additive White Gaussian Noise), considera-se que o ruído é gaussiano, aditivo e espacialmente branco. Sua matriz de covariância é dada por:

$$\mathbf{R}_n = \sigma^2 \mathbf{I}_M,$$

em que $\sigma^2$ representa a potência do ruído térmico por antena e $\mathbf{I}_M$ é a matriz identidade de dimensão $M \times M$. Como a matriz possui apenas elementos não nulos na diagonal, assume-se que os ruídos térmicos observados nas diferentes antenas são independentes. Nesse cenário, o equalizador LMMSE pode ser escrito como (JEON et al., 2019):

$$\mathbf{W}_{\text{LMMSE}} = \left(\mathbf{H}^H \mathbf{H} + \rho \mathbf{I}_K\right)^{-1} \mathbf{H}^H,$$

em que $\rho = \dfrac{\sigma^2}{E_s}$, $E_s$ representa a energia média dos símbolos transmitidos e $K$ é o número de usuários alvo.

Entretanto, em cenários com interferência, o ruído térmico não é a única contribuição presente no sinal recebido. Os sinais transmitidos por usuários que não são alvos da detecção também chegam às antenas da ~~estação base~~ BS. Como essas contribuições podem apresentar correção espacial, o ruído resultante deixa de ser branco. Neste trabalho, a combinação de interferência e ruído térmico é denominada ruído colorido.

#### *2.2.1.2 Assinatura espacial e Interferência*

O sinal transmitido por um ~~equipamento de usuário~~ UE chega as M antenas da ~~estação base~~ BS, multiplicado em cada antena pelo ganho do canal de propagação. Para um únco transmissor, o sinal recebido é:

$$\mathbf{y} = \mathbf{h}s + \mathbf{n},$$

em que em que $\mathbf{h} \in \mathbb{C}^{M\times 1}$ reúne os ganhos do canal entre o transmissor e as antenas da ~~estação base~~ BS. Esse vetor é denomidado assinatura espacial do sinal, porque descreve como o sinal de um usuário se distribui pelo conjunto de antenas.

Os UEs não-alvo da ~~estação base~~ BS também tem a sua assinatura espacial e o sinal de cada $r$ usuário interferente chega a todas as antenas, com ganhos distintos entre elas. Assim, a mesma interferência ocorre simultaneamente em todas as antenas, o que a torna correlacionada entre elas. O ruído térmico, por outro lado, é gerado de forma independente em cada antena.

Pode-se definir o modelo de sinal recebido no cenário de interferência como:

$$\mathbf{y} = \mathbf{H}\mathbf{s} + \mathbf{n} = \mathbf{H}\mathbf{s} + \overline{\mathbf{H}}\,\overline{\mathbf{s}} + \overline{\mathbf{n}},$$

em que $\mathbf{H} \in \mathbb{C}^{M\times K}$ é a matriz de canal dos $K$ usuários-alvo, $\mathbf{s} \in \mathbb{C}^{K\times 1}$ é o vetor de símbolos transmitidos por esses usuários, $\overline{\mathbf{H}} \in \mathbb{C}^{M\times r}$ representa os canais dos usuários interferentes e $\overline{\mathbf{s}} \in \mathbb{C}^{r\times 1}$ reúne seus símbolos transmitidos. O vetor $\overline{\mathbf{n}} \in \mathbb{C}^{M\times 1}$ representa o ruído térmico nas antenas receptoras.

$$\overline{\mathbf{H}} = \underbrace{\begin{bmatrix} \bar{h}_{11} & \cdots & \bar{h}_{1r} \\ \vdots & \ddots & \vdots \\ \bar{h}_{M1} & \cdots & \bar{h}_{Mr} \end{bmatrix}}_{M\times r}, \qquad \overline{\mathbf{s}} = \underbrace{\begin{bmatrix} \bar{s}_1 \\ \vdots \\ \bar{s}_r \end{bmatrix}}_{r\times 1}, \qquad \overline{\mathbf{n}} = \underbrace{\begin{bmatrix} \bar{n}_1 \\ \vdots \\ \bar{n}_M \end{bmatrix}}_{M\times 1}.$$

Para construir o modelo do ruído colorido, considera-se que os usuários interferentes transmitem de forma independente com potência média $\beta$, de modo que:

$$\mathbb{E}[\overline{\mathbf{s}}\,\overline{\mathbf{s}}^H] = \beta\mathbf{I}_r = \underbrace{\begin{bmatrix} \beta & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \beta \end{bmatrix}}_{r\times r}.$$

Os elementos em destaque na diagonal são a potência $\beta$ de cada interferente. Os zeros fora da diagonal indicam que os interferentes são independentes entre si.

Assume-se também que o ruído térmico é espacialmente branco, com média nula e matriz de covariância:

$$\mathbb{E}[\overline{\mathbf{n}}\,\overline{\mathbf{n}}^H] = \sigma^2\mathbf{I}_M = \underbrace{\begin{bmatrix} \sigma^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma^2 \end{bmatrix}}_{M\times M}.$$

Os elementos em destaque na diagonal são a potência $\sigma^2$ do ruído térmico em cada antena. Os zeros fora da diagonal indicam que o ruído de uma antena não depende do ruído das outras.

Além disso, os símbolos dos interferentes e o ruído térmico são considerados mutuamente independentes e de média nula. A matriz de canal $\overline{\mathbf{H}}$ é tratada como constante durante o intervalo de coerência considerado.

#### *2.2.1.3 Matriz de covariância do ruído*

O ruído colorido é definido pela soma das contribuições dos usuários interferentes e do ruído térmico:

$$\mathbf{n} = \overline{\mathbf{H}}\,\overline{\mathbf{s}} + \overline{\mathbf{n}}.$$

Sua matriz de covariância é dada por:

$$\mathbf{R}_n \triangleq \mathbb{E}[\mathbf{n}\mathbf{n}^H].$$

Como as duas parcelas são independentes e possuem média nula, a covariância da soma é igual à soma de suas covariâncias. Portanto,

$$\mathbf{R}_n = \mathrm{Cov}\left(\overline{\mathbf{H}}\,\overline{\mathbf{s}}\right) + \mathrm{Cov}(\overline{\mathbf{n}}).$$

A primeira parcela pode ser obtida pela propriedade da covariância sob uma transformação linear. Para uma matriz determinística $\mathbf{A}$ e um vetor aleatório $\mathbf{z}$, vale:

$$\mathrm{Cov}(\mathbf{A}\mathbf{z}) = \mathbf{A}\,\mathrm{Cov}(\mathbf{z})\mathbf{A}^H.$$

Aplicando essa propriedade à contribuição dos interferentes, obtém-se:

$$\begin{aligned}\mathrm{Cov}\left(\overline{\mathbf{H}}\,\overline{\mathbf{s}}\right) &= \overline{\mathbf{H}}\mathbb{E}[\overline{\mathbf{s}}\,\overline{\mathbf{s}}^H]\overline{\mathbf{H}}^H \\ &= \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H.\end{aligned}$$

Como a covariância do ruído térmico é $\sigma^2\mathbf{I}_M$, a matriz de covariância do ruído colorido resulta em:

$$\boxed{\mathbf{R}_n = \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I}_M}$$

$$\underbrace{\overline{\mathbf{H}}}_{M\times r}\,\underbrace{\beta\mathbf{I}_r}_{r\times r}\,\underbrace{\overline{\mathbf{H}}^H}_{r\times M} + \underbrace{\sigma^2\mathbf{I}_M}_{M\times M} = \underbrace{\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1M} \\ r_{21} & r_{22} & \cdots & r_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ r_{M1} & r_{M2} & \cdots & r_{MM} \end{bmatrix}}_{\mathbf{R}_n\ (M\times M)}$$

A matriz $\mathbf{R}_n$ possui dimensão $M \times M$ e reúne duas contribuições distintas. O termo $\sigma^2\mathbf{I}_M$ representa o ruído térmico, cuja covariância é nula entre antenas distintas. Já o termo $\beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H$ representa a covariância da interferência recebida. Como cada sinal interferente pode ser observado simultaneamente por várias antenas, essa parcela pode apresentar elementos não nulos fora da diagonal, indicando correlação espacial. Na matriz acima, esses são os elementos em destaque: cada um indica o quanto o ruído de duas antenas distintas está correlacionado. Os elementos da diagonal, sem destaque, são a potência do ruído em cada antena.

Assim, a matriz $\mathbf{R}_n$ descreve tanto a potência do ruído em cada antena quanto as relações estatísticas entre as diferentes componentes do vetor de ruído. Essa informação é necessária para modelar adequadamente a interferência no equalizador LMMSE. Sob a hipótese de símbolos-alvo independentes, com covariância $E_s\mathbf{I}_K$ e independentes do ruído, a matriz de covariância do sinal recebido fica:

$$\mathbf{R}_y = E_s\mathbf{H}\mathbf{H}^H + \mathbf{R}_n,$$

ou, equivalentemente,

$$\mathbf{R}_y = E_s\mathbf{H}\mathbf{H}^H + \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I}_M.$$

#### *2.2.1.4 Interferência sobre ruído térmico (IoT)*

A IoT mede a intensidade da interferência em relação ao ruído térmico. Ela é definida como (ZHAO et al., 2025):

$$\mathrm{IoT} = 10\log_{10}\left(\frac{\beta + N_0}{\sigma^2}\right), \tag{2.2}$$

em que $N_0$ é a potência do ruído térmico, ou seja, $N_0 = \sigma^2$, assim como na SNR medida por $E_s/N_0$ em (ZHAO et al., 2021).

A potência $\beta$ dos UEs não-alvo controla a intensidade da interferência. Quando a IoT é baixa, $\mathbf{R}_n$ se aproxima de $\sigma^2\mathbf{I}_M$ e o ruído é praticamente branco. À medida que a IoT cresce, a parcela $\beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H$ domina e a correlação entre as antenas se torna mais forte.

#### *2.2.1.5 Estimação da matriz de covariância*

A matriz $\mathbf{R}_n$ é uma grandeza teórica, definida por uma esperança, o que exigiria infinitas realizações do ruído, além de depender dos canais $\overline{\mathbf{H}}$ dos usuários interferentes e de sua potência de transmissão $\beta$ que o receptor não conhece. Por isso, $\mathbf{R}_n$ precisa ser estimada a partir de observações de amostras do ruído observadas pelo receptor.

A estimativa amostral é dada por:

$$\widehat{\mathbf{R}}_n = \frac{1}{N}\sum_{i=1}^{N}\mathbf{n}^i(\mathbf{n}^i)^H,$$

em que $N$ é o número de amostras disponíveis e $\mathbf{n}^i \in \mathbb{C}^{M\times 1}$ representa a $i$-ésima amostra de ruído colorido. Cada amostra corresponde a uma observação do ruído nas $M$ antenas da ~~estação base~~ BS.

As amostras de ruído são obtidas durante a transmissão de símbolos piloto. Como o sinal transmitido pelos usuários-alvo é conhecido pelo receptor nessa etapa, sua contribuição pode ser removida do sinal recebido. O resíduo contém a interferência dos usuários não-alvo somada ao ruído térmico, constituindo a amostra de ruído colorido utilizada na estimação.

O produto externo de uma amostra por sua transposta conjugada, $\mathbf{n}^i(\mathbf{n}^i)^H$, produz uma matriz de dimensão $M \times M$. Cada elemento dessa matriz corresponde ao produto entre o ruído observado em duas antenas na mesma amostra:

$$\left[\mathbf{n}^i(\mathbf{n}^i)^H\right]_{ab} = n_a^i(n_b^i)^*.$$

$$\underbrace{\begin{bmatrix} n_1^i \\ n_2^i \\ \vdots \\ n_M^i \end{bmatrix}}_{M\times 1} \underbrace{\begin{bmatrix} (n_1^i)^* & (n_2^i)^* & \cdots & (n_M^i)^* \end{bmatrix}}_{1\times M} = \underbrace{\begin{bmatrix} n_1^i(n_1^i)^* & n_1^i(n_2^i)^* & \cdots & n_1^i(n_M^i)^* \\ n_2^i(n_1^i)^* & n_2^i(n_2^i)^* & \cdots & n_2^i(n_M^i)^* \\ \vdots & \vdots & \ddots & \vdots \\ n_M^i(n_1^i)^* & n_M^i(n_2^i)^* & \cdots & n_M^i(n_M^i)^* \end{bmatrix}}_{M\times M}$$

Quando $a = b$, o produto corresponde à potência observada na antena considerada. Esses são os elementos em destaque na diagonal da matriz acima. Quando $a \neq b$, o produto relaciona os sinais de ruído observados em duas antenas distintas.

A média desses produtos ao longo das $N$ amostras fornece uma estimativa dos elementos correspondentes de $\mathbf{R}_n$. Dessa forma, cada amostra contribui para a estimação da potência do ruído em cada antena e das relações estatísticas entre pares de antenas. Uma única observação não é suficiente para caracterizar essas propriedades estatísticas; a estimativa torna-se mais representativa à medida que são incorporadas novas amostras. Ou seja, a qualidade da estimativa depende do número de amostras utilizadas.

A quantidade de amostras, no entanto, é limitada pelos recursos disponíveis para a coleta. No sistema considerado, as amostras são obtidas a partir de elementos de recurso de pilotos, cuja disponibilidade depende da configuração do sinal e do intervalo durante o qual a covariância pode ser considerada constante.

#### *2.2.1.6 Availability of noise samples within the coherence time*

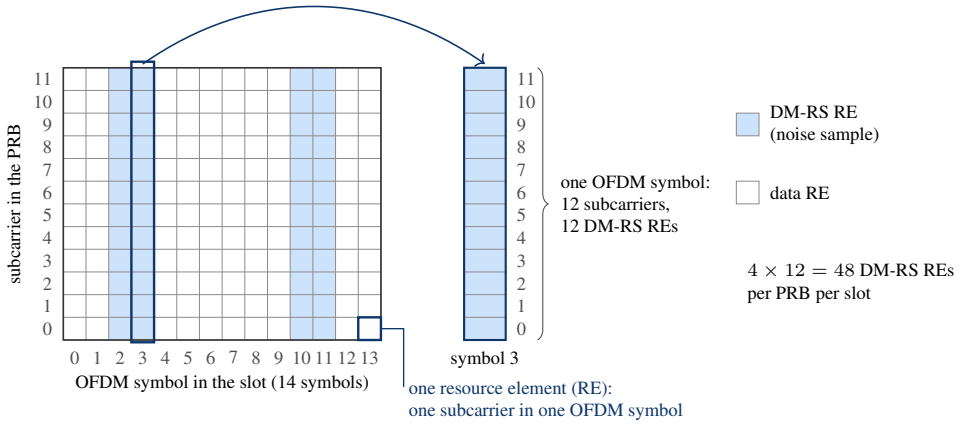
All samples used in noise estimation must represent the same covariance matrix. Therefore, the sampling interval is limited by the variation in time and frequency of the channels that determine the interference. The variation in time is defined by the coherence time. The variation in frequency is defined by the PRB. It is considered that the channels of the interfering users remain constant within the PRB (ZHAO et al., 2025). The estimate $R_{cc}$, therefore, is unique per PRB within each coherence time.

In 5G NR, the Demodulation Reference Signal (DM-RS) is the reference signal transmitted by the UE to allow the receiver to estimate channel conditions. The DM-RS is transmitted alongside the PUSCH (Section 2.1.1) within the same PRBs (3GPP, 2025, Sec. 6.4.1.1.3). Each UE transmits its own DM-RS within the same OFDM symbols, and the receiver must distinguish the DM-RS of each one. The double-symbol DM-RS occupies two consecutive OFDM symbols at each position within the slot and allows

for the differentiation of up to 12 UEs (3GPP, 2025, Tab. 6.4.1.1.3-5). To achieve this, each UE employs different subcarriers or sequences of $+1$ and $-1$ values (3GPP, 2025, Tab. 6.4.1.1.3-2). For instance, when two UEs utilize the same subcarriers, one may transmit $(+1, +1)$ across two adjacent REs while the other transmits $(+1, -1)$; the sum and difference of the received REs then allow for the isolation of each UE.

Noise samples are obtained from the REs occupied by the DM-RS. Como o DM-RS só é transmitido junto com o PUSCH, só há amostras nos slots em que os UEs alvo transmitem o PUSCH (Seção 2.1.2). The number of samples therefore depends on how many DM-RS REs are present within the interval over which the covariance remains constant. With a maximum of two positions per slot, the double-symbol DM-RS occupies 4 OFDM symbols (3GPP, 2025, Tab. 6.4.1.1.3-4). Since each PRB contains 12 subcarriers (Section 2.1.2), a maximum of 4 $\times$ 12 = 48 DM-RS REs per slot are available within each PRB. Figure 2.1 illustrates these REs within a PRB. Consequently, the quantity of samples collected within an interval $T_c$ depends on the number of OFDM symbols contained within that interval.

Figure 2.1: Resource elements of one PRB in one slot, with the DM-RS symbols.




Source: the author, with the DM-RS positions of 3GPP, 2025. Double-symbol DM-RS, mapping type A, symbols 2, 3, 10 and 11.

A sample represents the noise covariance for a given data symbol only if both fall within the same interval of duration $T_c$. As a result, the number of available samples may vary from one data symbol to another. For any given symbol, the number of samples corresponds to the maximum count of DM-RS REs found within an interval of duration $T_c$ that encompasses that symbol. The number of samples available for all symbols in the slot is the smallest of these values. Figure 2.2 shows an example with the DM-RS located at symbols 2, 3, 10, and 11 of each slot. Under the TDLB100-400 condition, symbols 5 through 8 represent the worst-case scenario, with 24 samples. Therefore, the

number of samples available for all symbols is 24. Table 2.3 presents this value for each test condition.

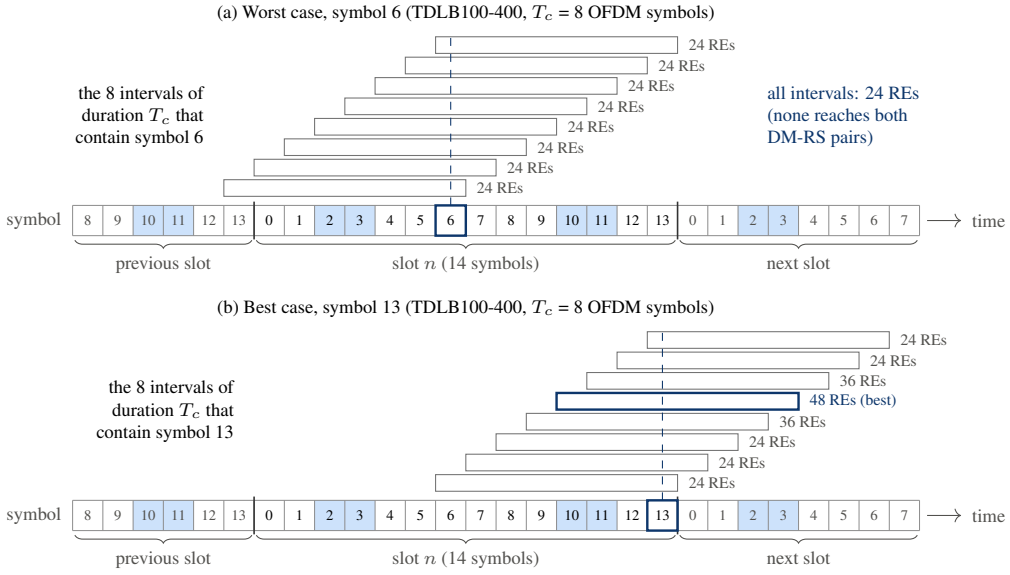
Table 2.3: Number of DM-RS REs per PRB available to every data symbol of the slot, in each test condition.

| **Test condition** | **Frequency range** | $T_c$ **(ms)** | **OFDM symbols in** $T_c$ | **DM-RS REs available** |
|---|---|---|---|---|
| TDLA30-5 | FR1 | 25.0000 | 700 | 2400 |
| TDLA30-10 | | 12.5000 | 350 | 1200 |
| TDLA30-75 | FR2 | 1.6667 | 186 | 648 |
| TDLA30-300 | | 0.4167 | 46 | 168 |
| TDLC300-100 | FR1 | 1.2500 | 35 | 132 |
| TDLB100-400 | | 0.3125 | 8 | 24 |
| HST350 | | 0.0536 | 1 | 0 |
| HST500 | | 0.0375 | | |

Source: the author, from 3GPP, 2020; 3GPP, 2025; TSE; VISWANATH, 2005. Double-symbol DM-RS in symbols 2, 3, 10 and 11, with every slot carrying PUSCH.

The best range of a data symbol may include DM-RS that arrive after that symbol. In Figure 2.2(b), the best range of symbol 13 goes up to symbol 3 of the next slot. To use these samples, the receiver needs to store symbol 13 for 4 more symbols.

Figure 2.2: Intervals of duration $T_c$ that contain a data symbol, and their DM-RS REs per PRB, in the TDLB100-400 condition.




Source: the author, with the DM-RS positions of 3GPP, 2025. Double-symbol DM-RS, mapping type A, symbols 2, 3, 10 and 11; continuous uplink. In (a) and (b), each bar is one position of an interval of duration $T_c$ that contains the marked symbol. Symbol 6 is the worst case of the slot, and symbol 13 is a best case.

*2.2.1.7 Condicionamento da estimativa*

O posto de uma matriz é o número máximo de linhas ou colunas linearmente independentes que essa matriz possui. Cada parcela $\mathbf{n}^i(\mathbf{n}^i)^H$ é uma matriz de posto unitário, uma vez que todas as suas colunas são múltiplas do mesmo vetor $\mathbf{n}^i$. Consequentemente, o posto da soma de (N) desses produtos é limitado por (N):

$$\operatorname{rank}(\widehat{\mathbf{R}}_n) \leq \min(M, N)$$

Assim, quando (N<M), a estimativa amostral da matriz completa $(M \times M)$ é necessariamente singular. Uma matriz singular não é inversível. Essa propriedade precisa ser considerada em operações que exigem a inversão da matriz de covariância.

(ZHAO et al., 2025) adota a condição $N \gg K$, para obter uma estimativa suficientemente precisa. Em suas simulações, utiliza (N=192) amostras por realização do canal. Um trabalho anterior dos mesmos autores também apresenta 96 e 192 como valores típicos para essa quantidade de amostras (Zhao et al., 2021).

Uma forma de tornar a estimativa inversível é o encolhimento (*shrinkage*) de Ledoit e Wolf. Ele troca $\hat{\mathbf{R}}_n$ por uma média ponderada entre $\hat{\mathbf{R}}_n$ e um múltiplo da identidade (LEDOIT; WOLF, 2004):

$$\hat{\mathbf{R}}_{\text{LW}} = (1-\rho)\,\hat{\mathbf{R}}_n + \rho\,\mu\,\mathbf{I}_M, \qquad \mu = \frac{\operatorname{tr}(\hat{\mathbf{R}}_n)}{M}. \tag{2.3}$$

O peso $\rho$, entre 0 e 1, é calculado a partir das próprias amostras, de forma a minimizar o erro quadrático esperado entre a estimativa e a covariância verdadeira (LEDOIT; WOLF, 2004). Assim, não é preciso escolher uma constante. Como $\hat{\mathbf{R}}_n$ não tem autovalores negativos, todo autovalor de $\hat{\mathbf{R}}_{\text{LW}}$ é pelo menos $\rho\mu$. Quando $\rho > 0$, a matriz resultante é inversível.

### 2.2.2 Processamento descentralizado

Na arquitetura descentralizada, as $M$ antenas da ~~estação base~~ BS são particionadas em $C$ clusters, cada um associado a uma DU (*Distributed Unit*), uma unidade de processamento local (ZHAO et al., 2025). O vetor de ruído pode, então, ser organizado em componentes locais:

$$\mathbf{n} = \begin{bmatrix} \mathbf{n}_1 \\ \mathbf{n}_2 \\ \vdots \\ \mathbf{n}_C \end{bmatrix}.$$

Essa partição induz uma estrutura em blocos na matriz de covariância $\mathbf{R}_n$. Os blocos diagonais $\mathbf{R}_{cc}$ descrevem a covariância do ruído observado pelas antenas do cluster $c$, enquanto os blocos fora da diagonal, $\mathbf{R}_{mn}$, representam as correlações entre antenas pertencentes a clusters distintos.

Cada DU dispõe localmente das amostras de ruído de suas próprias antenas. Assim, a DU $c$ pode estimar seu bloco diagonal sem trocar dados com as demais unidades:

$$\hat{\mathbf{R}}_{cc} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{n}_c^i (\mathbf{n}_c^i)^H.$$

Os blocos fora da diagonal, por outro lado, dependem de amostras provenientes de duas ou mais DUs. Sua estimação direta exige a troca de informações entre as unidades, aumentando o tráfego de comunicação.

Essa limitação estabelece uma relação entre as propriedades estatísticas do ruído e a estratégia de equalização descentralizada.

### 2.2.3 Equalizador BCD-MMSE

O equalizador BCD-MMSE (*Block Coordinate Descent–Minimum Mean Square Error*) (ZHAO et al., 2025), por sua vez, emprega atualizações distribuídas para tratar o acoplamento entre os blocos do sistema, mantendo a necessidade de definir quais informações devem ser compartilhadas entre as unidades.

No BCD-MMSE considerado neste trabalho, a estimativa local $\hat{\mathbf{R}}_{cc}$ participa da inicialização do equalizador e da construção da matriz utilizada nas atualizações da DU. Em particular, a matriz local a ser invertida é definida por:

$$\mathbf{G}_c = E_s \mathbf{H}_c \mathbf{H}_c^H + \hat{\mathbf{R}}_{cc},$$

em que $\mathbf{H}_c$ representa a parcela da matriz de canal associada ao cluster $c$. Dessa forma, a estimação e o condicionamento da covariância local afetam diretamente a via-

bilidade das operações matriciais necessárias à equalização.

A DU $c$ usa $\hat{\mathbf{R}}_{cc}$ de duas formas. Na inicialização, usa a inversa $\hat{\mathbf{R}}_{cc}^{-1}$, pois o BCD-MMSE começa pela aproximação bloco-diagonal (ZHAO et al., 2025). Nas iterações, usa $\mathbf{G}_c$, que soma $\hat{\mathbf{R}}_{cc}$ ao termo do canal (ZHAO et al., 2025). Assim, $\mathbf{G}_c$ usa $\hat{\mathbf{R}}_{cc}$, e não a sua inversa. Além disso, $\mathbf{G}_c$ não depende da iteração (ZHAO et al., 2025). Ela é formada e invertida uma vez, e a inversa é usada em todas as iterações.

O número $N$ de amostras também tem um custo de armazenamento e de comunicação. Cada DU guarda as $N$ amostras de ruído das suas antenas (ZHAO et al., 2025). Trocar essas amostras entre as DUs exigiria transferir dados de tamanho $M \times N$ (ZHAO et al., 2025). No BCD-MMSE, a banda de comunicação em cada iteração também depende de $N$ (ZHAO et al., 2025). Como a estimativa só muda a cada bloco de coerência, esse custo é pago uma vez a cada $N_{\text{coh}}$ símbolos (ZHAO et al., 2025).

#### *2.2.3.1 Complexidade*

No artigo base, o pré-processamento de cada DU tem complexidade $\mathcal{O}(M_c^3 + NM_c^2)$, e cada iteração do BCD-MMSE tem complexidade $\mathcal{O}(NMK + MM_cK)$ no total (ZHAO et al., 2025). A Tabela 2.4 detalha o número de multiplicações complexas de cada parte do algoritmo em uma DU, por PRB e por $T_c$, para $K = 8$ e ~~$N = 192$~~. Como $N$ muda com a condição de teste, a tabela mostra o maior e o menor valor usados, $N = 192$ e $N = 24$ (Tabela 3.3). As linhas que não dependem de $N$ têm um só valor.

Table 2.4: Multiplicações complexas por PRB e por $T_c$ em cada DU.

| Parte | Conta | $M_c = 16$ | | $M_c = 8$ | | Onde rod |
|---|---|---|---|---|---|---|
| | | $N = 192$ | $N = 24$ | $N = 192$ | $N = 24$ | |
| Estimar $\hat{\mathbf{R}}_{cc}$ e aplicar Ledoit-Wolf | $NM_c^2 + NM_c + M_c^2$ | 52480 | 6784 | 13888 | 1792 | unidade |
| Inicialização | $M_c^2K + M_cK^2 + 2K^2M_c + NKM_c$ | 29696 | 8192 | 14336 | 3584 | unidade |
| Formar $\mathbf{G}_c$ | $M_c^2K$ | 2048 | | 512 | | unidade |
| Cada iteração | $2NKM_c + 3K^2M_c + KM_c^2$ | 54272 | 11264 | 26624 | 5120 | unidade |
| Inverter $\hat{\mathbf{R}}_{cc}$ | $M_c^3$ | 4096 | | 512 | | módulo 1 |
| Inverter $\mathbf{G}_c$ | $M_c^3$ | 4096 | | 512 | | módulo 2 |
| Inverter matriz $K \times K$ (só na DU $C$) | $K^3$ | 512 | | 512 | | módulo 3 |
| Filtro, por símbolo de dados | $12KM_c$ | 1536 | | 768 | | unidade |

## 2.3 Inversão de Matrizes

### 2.3.1 Inversao de Matrizes

#### *2.3.1.1 Decomposicao de Cholesky*

#### *2.3.1.2 Representação em ponto fixo*

Em hardware, os números costumam ser representados em ponto fixo. No Vitis HLS, o tipo `ap_fixed`$\langle W, I\rangle$ tem $W$ bits no total, dos quais $I$ são da parte inteira (KASTNER; MATAI; NEUENDORFFER, 2018, p. 73–74). A parte fracionária tem $F = W - I$ bits. Neste trabalho, o formato é escrito como $\langle W, I\rangle$.

Quando um valor não cabe na parte inteira, ocorre um estouro (*overflow*). Uma forma comum de tratá-lo é descartar os bits mais significativos, o que se chama *wrapping* (KASTNER; MATAI; NEUENDORFFER, 2018, p. 70). Em complemento de dois, isso pode até trocar o sinal do número (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71). Quando um valor não cabe na parte fracionária, ele precisa ser arredondado (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71). Descartar os bits extras arredonda em direção a $-\infty$. Outras formas arredondam em direção a $+\infty$, em direção a zero, para longe de zero, ou para o valor mais próximo, com o empate indo para o valor par (KASTNER; MATAI; NEUENDORFFER, 2018, p. 71).

O tipo `ap_fixed` tem 7 modos de quantização e 5 modos de estouro, o que dá 35 combinações. A Tabela 2.5 descreve cada um.

#### *2.3.1.3 Latência, intervalo e vazão*

Em síntese de alto nível, uma tarefa é uma chamada de função. A latência da tarefa é o tempo entre o início e o fim de uma tarefa. O intervalo da tarefa é o tempo entre o início de uma tarefa e o início da seguinte (KASTNER; MATAI; NEUENDORFFER, 2018, p. 20). Em um circuito com *pipeline*, uma nova tarefa pode começar antes de a anterior terminar. Assim, o intervalo pode ser menor que a latência (KASTNER; MATAI; NEUENDORFFER, 2018, p. 21). O intervalo entre dois dados que entram no *pipeline* é chamado de intervalo de iniciação (II, *Initiation Interval*) (KASTNER; MATAI; NEUENDORFFER, 2018, p. 21).

No começo, o *pipeline* está vazio, e só os primeiros estágios trabalham. O tempo até todos os estágios ficarem ocupados é o tempo de enchimento. No fim, o *pipeline* tam-

Table 2.5: Modos de quantização e de estouro do tipo `ap_fixed`.

| Modo | Comportamento |
|---|---|
| *Quantização (afeta os bits fracionários)* | |
| `AP_TRN` | Descarta os bits menos significativos. Arredonda em direção a $-\infty$. |
| `AP_TRN_ZERO` | Arredonda em direção a zero. Para números positivos, é igual ao `AP_TRN`. |
| `AP_RND` | Arredonda para o valor mais próximo. No empate, arredonda em direção a $+\infty$. |
| `AP_RND_ZERO` | Arredonda para o valor mais próximo. No empate, arredonda em direção a zero. |
| `AP_RND_MIN_INF` | Arredonda para o valor mais próximo. No empate, arredonda em direção a $-\infty$. |
| `AP_RND_INF` | Arredonda para o valor mais próximo. No empate, afasta-se de zero. |
| `AP_RND_CONV` | Arredonda para o valor mais próximo. No empate, vai para o valor par. |
| *Estouro (afeta os bits inteiros)* | |
| `AP_SAT` | Satura no maior ou no menor valor representável. |
| `AP_SAT_ZERO` | Em caso de estouro, o resultado vira zero. |
| `AP_SAT_SYM` | Satura de forma simétrica: os limites positivo e negativo têm o mesmo módulo. |
| `AP_WRAP` | Descarta os bits mais significativos que não cabem (estouro em complemento de dois). |
| `AP_WRAP_SM` | Faz o mesmo descarte, mas em sinal e magnitude. |

Fonte: TODO (UG1399).

bém precisa esvaziar. Esse custo fixo pesa mais quando há poucas iterações (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 114 e 146).

Em uma sequência de tarefas, a primeira termina depois da latência $L$, e cada uma das seguintes, um intervalo $I$ depois da anterior. Assim, $n$ tarefas terminam em

$$t_n = L + (n-1)\,I. \tag{2.4}$$

Muitas ferramentas chamam o intervalo de vazão (*throughput*). Um intervalo maior quer dizer menos tarefas por unidade de tempo, ou seja, uma vazão menor (KASTNER; MATAI; NEUENDORFFER, 2018, p. 22). Neste trabalho, a vazão é medida pelo intervalo entre inversões, e a latência é o tempo de uma inversão.

#### *2.3.1.4 Pipeline de tarefas*

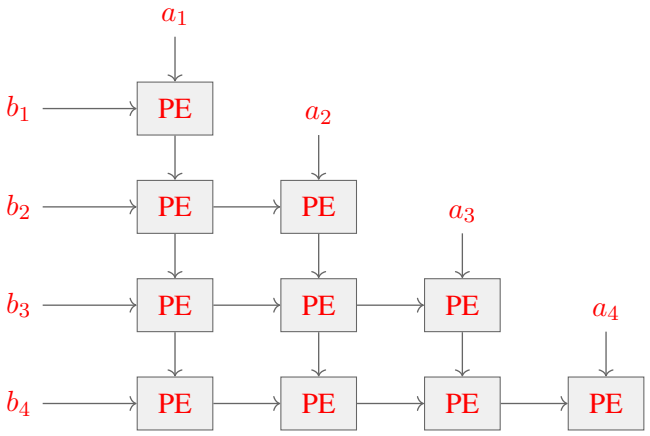
Uma tarefa pode ser dividida em etapas. No *pipeline* de tarefas, as etapas trabalham ao mesmo tempo, cada uma em um dado diferente. Assim, a tarefa seguinte começa quando a anterior termina a primeira etapa. Em HLS, isso é feito com a diretiva `dataflow`, e cada etapa vira um processo separado. O intervalo do conjunto nunca é menor que o intervalo da etapa mais lenta (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 113–115).

#### *2.3.1.5 Arranjo sistólico*

Um arranjo sistólico é um conjunto de elementos de processamento (PEs, *processing elements*) iguais, em que cada PE só troca dados com os vizinhos. Esse tipo de projeto pode surgir quando um laço interno é desenrolado: cada cópia do corpo do laço vira um PE (KASTNER; MATAI; NEUENDORFFER, 2018, p. 180).

A Figura 2.3 mostra um arranjo sistólico em grade que calcula a matriz de Gram de $\mathbf{H}$. Como essa matriz é hermitiana, só o triângulo inferior é calculado. A cada ciclo, o arranjo lê uma linha de $\mathbf{H}$. Os dados entram por cima e pela esquerda e passam para o PE de baixo e para o da direita. Cada PE multiplica as duas entradas e acumula o produto. Depois de ler todas as linhas, cada PE guarda um elemento da matriz (WANG et al., 2021, p. 2979).

Figure 2.3: Lower triangular systolic array that computes a Gram matrix.




Source: adapted from WANG et al., 2021, p. 2979. $a_j$ and $b_i$: elements of one row of $\mathbf{H}$, one row per cycle. Each PE multiplies $a_j$ by $b_i$, adds the product to its accumulator, and passes $a_j$ down and $b_i$ to the right.

# 3 METHODOLOGY

This chapter describes the tools used and the tests performed for this work. The uplink transmission of a massive MIMO system with decentralized processing (Section 2.1.1) was simulated to analyze the effect of the approximation used in the inversion module of the BCD-MMSE algorithm. Based on this analysis, the fixed-point formats for the inversion module, implemented on an FPGA, are selected. The chapter begins by presenting the simulation workflow (Section 3.1) and the hardware tools (Section 3.2). It then covers test configurations and conditions (Section 3.3), timing constraints for the inversion process (Section 3.4), the selection of the numerical format (Section 3.5), the hardware architectures, the matrices used to test them and the power estimation (Section 3.6), and the organization of the results (Section 3.7).

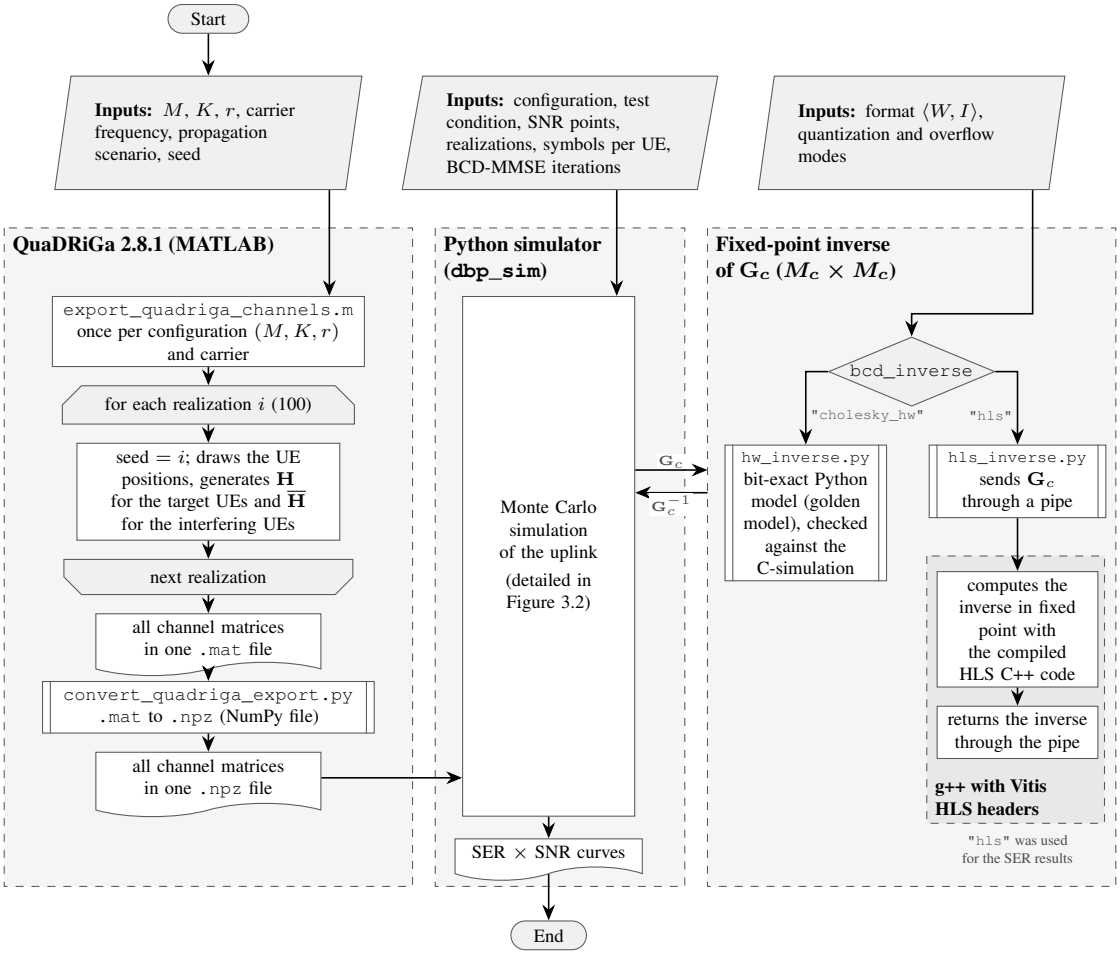
## 3.1 Simulation workflow

The simulation workflow 3.1 employs three tools: the QuaDRiGa channel generator, a Python simulator developed in this work, and C++ code developed in Vitis HLS, which computes the inverse of $\mathbf{G}_c$ using fixed-point arithmetic. QuaDRiGa generates the channel matrices $\mathbf{H}$ for the target UEs and $\overline{\mathbf{H}}$ for the interfering UEs.

The Python simulator handles the transmission process. It generates symbols for the target UEs and creates colored noise, with a covariance derived from $\overline{\mathbf{H}}$ (Section 2.2.1), to construct the received signal using $\mathbf{H}$. Subsequently, it computes the equalizers, performs symbol detection, and measures the SER at various SNR levels. The BCD-MMSE algorithm is simulated in two ways: using the inverse of $\mathbf{G}_c$ in 64-bit floating-point format (which serves as a reference) and using the inverse in fixed-point format. In the latter case, whenever the BCD-MMSE algorithm requires $\mathbf{G}_c^{-1}$, the Python simulator invokes the C++ code.

The Python simulator models the same uplink as in (ZHAO et al., 2025). According to the authors, that paper was the first work on decentralized processing to consider colored noise (ZHAO et al., 2025). Therefore, the simulation parameters follow that work, except where the text indicates otherwise.

Figure 3.1: Simulation flow: QuaDRiGa channels, Python simulator and fixed-point inverse.




Source: the author.

### 3.1.1 Channel generation

The channels are generated using QuaDRiGa 2.8.1 in MATLAB. They depend on the number of antennas $M$, the number of target UEs $K$, the number of interfering UEs $r$, and the carrier frequency. Consequently, they are generated once for each combination of these values. Other parameters, such as $C$, the modulation scheme, and the IoTh, only affect the processing steps within the Python-based simulator. The simulation employs 100 channel realizations. Each realization consists of a random selection of UE positions, along with the corresponding channels $\mathbf{H}$ and $\overline{\mathbf{H}}$ generated by QuaDRiGa for those specific positions. Every realization uses its own unique random seed.

MATLAB saves the channels for all 100 realizations into a `.mat` file. A Python script then converts this file into an `.npz` file, NumPy's format for storing multiple arrays within a single compressed file. The Python-based simulator reads the matrices $\mathbf{H}$ and $\overline{\mathbf{H}}$ from this file.

### 3.1.2 Python simulator

The Python simulator runs two nested loops (Figure 3.2). The outer loop iterates through the 100 realizations, while the inner loop iterates through the SNR points. The SNR points range from $-10$ to $10$ dB, with a step of $2$ dB. For each realization, the simulator takes $\mathbf{H}$ and $\overline{\mathbf{H}}$ corresponding to that realization and uses these channels across all SNR points. Each point on the SER curves represents the average over the 100 realizations. This is a Monte Carlo simulation, as in (ZHUANG et al., 2025): the SER (Section 2.1.5.6) is estimated by the fraction of erroneous symbols across many random draws of the channel, noise, and symbols.

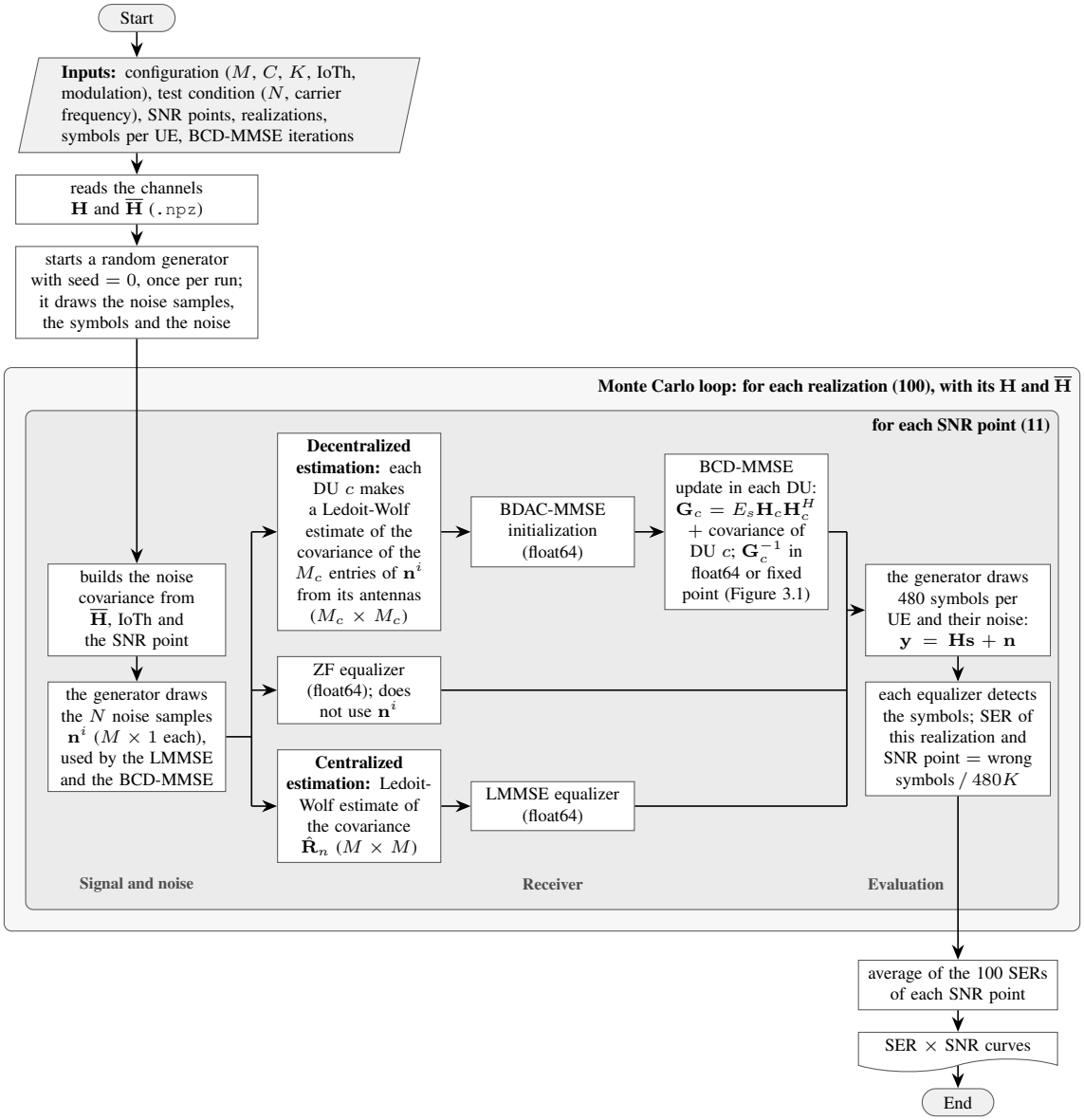
At each SNR point, the simulator constructs the noise covariance matrix based on $\overline{\mathbf{H}}$, the IoTh (interference-over-thermal), and the SNR. It then generates $N$ noise samples, $\mathbf{n}^i$ (Section 2.2.1.5). It also generates 480 data symbols for each UE, along with the noise associated with these symbols. Based on this, the equalizers are calculated. The channel remains constant within the coherence block (Section 2.1.4.3); therefore, each equalizer is computed once and applied to all 480 symbols. The BCD-MMSE is compared against two centralized equalizers, the LMMSE and the ZF, which serve as benchmarks. The ZF does not utilize the noise samples $\mathbf{n}^i$. In contrast, both the LMMSE and the BCD-MMSE employ these samples to estimate the noise covariance (Section 2.2.1.7). This estimate is always subjected to Ledoit–Wolf shrinkage (Equation 2.3) before being used, even in cases where the estimate is already invertible.

In the LMMSE, this adjustment is applied to the estimate $\hat{\mathbf{R}}$ corresponding to all $M$ antennas. In the BCD-MMSE, each DU performs the adjustment on its own block, $\hat{\mathbf{R}}_{cc}$, using only the samples from its $M_c$ antennas. Consequently, no DU requires access to the samples from any other DU. The adjusted block forms the matrix $\mathbf{G}_c$ (Section 2.2.3). In the reference version of the BCD-MMSE, this matrix is inverted using 64-bit floating-point arithmetic; alternatively, it can be inverted using fixed-point arithmetic (Section 3.1.3).

### 3.1.3 Fixed-point inverse

The inverse of $\mathbf{G}_c$ in fixed-point arithmetic is derived from the C++ code of the inversion module (Section 3.2.2). This code is compiled using `g++` along with the `ap_fixed` header files from Vitis HLS 2026.1, with a separate compilation performed

Figure 3.2: Detailed flow of the Python simulator: Monte Carlo loop, equalizers and SER.




Source: the author.

for each combination of format $\langle W, I \rangle$, quantization mode, overflow mode (Table 2.5), and size $M_c$. The program remains active throughout the simulation. Other calculations within the simulator, including the inverses used for BDAC initialization and LMMSE, utilize 64-bit floating-point arithmetic.

To verify this program, the same $\mathbf{G}_c$ matrices used in the simulator were inverted both by this program and by the Vitis HLS C-simulation. This process involved 30 matrices for each of the 35 mode combinations, with $M_c$ set to 16 and 32. In every instance, the resulting inverses were bit-identical. Consequently, the Symbol Error Rate (SER) is measured using the exact same inverse as that calculated by the hardware. The simulator also

includes a Python-based model of the inversion, which has been verified bit-for-bit against the C++ code; however, the reported SER results are based on the C++ implementation.

## 3.2 Hardware setup

### 3.2.1 Tools and target device

The inversion module was described in C++ and synthesized in Vitis HLS 2026.1. Vitis HLS generates the module's RTL. Then, the RTL was synthesized, placed, and routed in Vivado 2023.2. The target device is the AMD Alveo U50 board, with the XCU50-FSVH2104-2-E FPGA. This FPGA has 5,952 DSP blocks and 871,680 LUTs. All circuits use a clock frequency of 100 MHz, or a period of 10 ns.

In Vivado, the inversion module is synthesized and implemented alone, as the top-level design, without any other surrounding board logic. The flow performs synthesis, optimization, placement, physical optimization, and routing. The only timing constraint is the clock period, and the paths entering and exiting the module's ports are not constrained. Thus, the resource, timing, and power figures for this work are only for the module and come from the routed design in Vivado 2023.2.

Before the Vivado workflow, each version went through the Vitis HLS C synthesis. A version only went to Vivado if the HLS estimate fit within the FPGA: a maximum of 5,952 DSP blocks and a maximum of 90% of the LUTs.

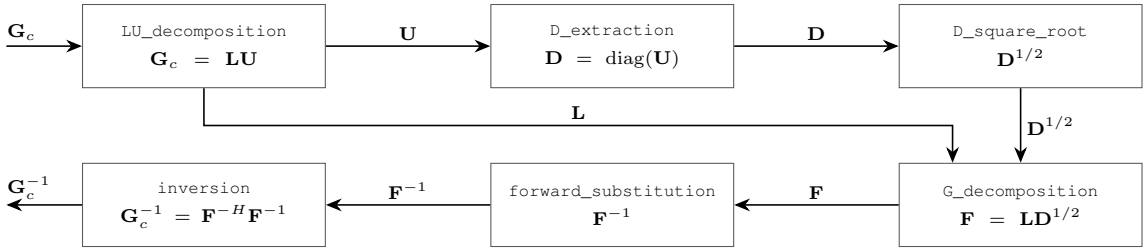
### 3.2.2 Inversion module

The module receives the $M_c \times M_c$ matrix $\mathbf{G}_c$ and returns its inverse. It calculates the inverse using Cholesky decomposition (Section 2.3). Two sizes for $\mathbf{G}_c$, corresponding to the configurations described in Section 3.3.1, have been implemented: $16 \times 16$ and $32 \times 32$. The input, output, and internal values use the `ap_fixed` data type, following the formats selected in Section 3.5.

The module consists of six stages, each implemented as a C++ function (Figure 3.3). These stages are executed sequentially, with each stage passing its output to the next.

Additions, multiplications, and divisions utilize `ap_fixed` type operators. Com-

Figure 3.3: Steps of the inversion module. Each block is one C++ function of the HLS code.




Source: the author.

plex multiplication and division operations are explicitly implemented in the code, with a specific type defined for each intermediate value; thus, the bit-width of every intermediate value is explicitly set within the code. The addition of two $N$-bit numbers requires $N + 1$ bits, while their product requires $2N$ bits (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 72–73). When using `ap_fixed` types, the result of an operation inherently possesses a sufficient number of bits; overflow only occurs when this result is assigned to a type with a smaller bit-width (KASTNER; MATAI; NEUENDORFFER, 2018, p. 74). Consequently, intermediate values employ types with greater bit-widths than the $\langle W, I\rangle$ format (Table 3.1), with the result reverting to the specified format only at the output of each stage. Square root calculations are performed using the `hls::sqrt` function from Vitis HLS, operating directly within the defined format.

Table 3.1: Types of the values in the inversion module, for the format $\langle W, I\rangle$ with $F = W - I$ fractional bits.

| **Value** | **Integer bits** | **Fractional bits** |
|---|---|---|
| Input, output and output of each step | $I$ | $F$ |
| Square root (`hls::sqrt`) | $I$ | $F$ |
| Accumulated sums, numerator and divisor of the complex division, quotient | $2I$ | $2F$ |
| Product of two real numbers | $4I$ | $4F$ |
| Real or imaginary part of a complex product (sum of two products) | $4I + 1$ | $4F$ |

Source: the author. The intermediate values use the same quantization and overflow modes as the format; the square root uses AP_TRN and AP_WRAP.

## 3.3 Configurations and test conditions

### 3.3.1 Base station configurations

The uplink of the massive MIMO system was investigated using two types of parameters. Hard constraints refer to the physical characteristics of the base station: $M = 128$ or 256 antennas, $C = 8$ or 16 clusters, and the size $M_c = M/C$ of the inversion module. Soft constraints depend on the UEs being served and the level of interference: $K = 8$ or 12 UEs, QPSK or 16-QAM modulation, and an IoTh ratio of 10 dB as a baseline case, or 20 dB representing an extreme, high-interference scenario. These configurations were adapted from (ZHAO et al., 2025). Table 3.2 presents these configurations.

Each configuration is identified by a label consisting of a number and a letter. The number denotes the hard constraints: 1 for $M = 128$ and $C = 8$; 2 for $M = 256$ and $C = 8$; and 3 for $M = 256$ and $C = 16$. The letter indicates the soft constraints and remains consistent across all three groups.

### 3.3.2 Test conditions

The simulated test conditions of Table 2.3 define the carrier frequency and the number of noise samples in the Python simulator, as shown in Table 3.3. The simulation does not include a time axis, and the channel does not account for the Doppler effect, so only these two variables change between test conditions. A frequency of 3.5 GHz was used for FR1 conditions, within the n78 band (3300 to 3800 MHz, TDD), and 28 GHz was used for FR2 conditions, within the n257 band (26500 to 29500 MHz, TDD) (3GPP, 2020, Tabs. 5.2-1 and 5.2-2). In each realization, the channels at both frequencies use the same UE positions.

This work employs a double-symbol DM-RS, located at symbols 2, 3, 10, and 11 of each slot (3GPP, 2025, Tab. 6.4.1.1.3-4), as it enables the differentiation of DM-RS signals among $K = 8$ or 12 UEs (Section 2.2.1.6). The value of $N$ for each condition is taken from Table 2.3, with an upper limit of 192, the value used in (ZHAO et al., 2025). This $N$ assumes that the receiver waits for samples arriving after the data symbol (Section 3.4.1). The HST350 and HST500 conditions were not simulated, as they correspond to $N = 0$.

TDLA30-5 and TDLA30-10 share the same carrier frequency and the same $N$

Table 3.2: Simulated configurations.

| **Configuration** | **Hard constraints** | | | **Soft constraints** | | |
|---|---|---|---|---|---|---|
| | $M$ | $C$ | $M_c$ | **IoTh (dB)** | **Modulation** | $K$ |
| – | M | C | – | iot_db | modulation | K |
| 1A | 128 | 8 | 16 | 10 | 16-QAM | 8 |
| 1B | | | | | | 12 |
| 1C | | | | | QPSK | 8 |
| 1D | | | | | | 12 |
| 1E | | | | 20 | 16-QAM | 8 |
| 1F | | | | | | 12 |
| 1G | | | | | QPSK | 8 |
| 1H | | | | | | 12 |
| 2A | 256 | 8 | 32 | 10 | 16-QAM | 8 |
| 2B | | | | | | 12 |
| 2C | | | | | QPSK | 8 |
| 2D | | | | | | 12 |
| 2E | | | | 20 | 16-QAM | 8 |
| 2F | | | | | | 12 |
| 2G | | | | | QPSK | 8 |
| 2H | | | | | | 12 |
| 3A | 256 | 16 | 16 | 10 | 16-QAM | 8 |
| 3B | | | | | | 12 |
| 3C | | | | | QPSK | 8 |
| 3D | | | | | | 12 |
| 3E | | | | 20 | 16-QAM | 8 |
| 3F | | | | | | 12 |
| 3G | | | | | QPSK | 8 |
| 3H | | | | | | 12 |

Source: 1A, 1B, 1C, 1E, 2A and 3A adapted from ZHAO et al., 2025; the others, the author. $r = 8$ in all configurations. Second header row: field names of `Panel` in the code.

Table 3.3: Carrier frequency and number of noise samples in each test condition.

| **Test condition** M1_Driver. test_conditions | **Frequency range** – | **Carrier frequency (GHz)** M1_Driver. test_conditions | **Noise samples $N$** M5_Noise. n_noise_samples |
|---|---|---|---|
| TDLA30-5 | FR1 | 3.5 | 192 |
| TDLA30-10 | FR1 | 3.5 | 192 |
| TDLA30-75 | FR2 | 28 | 192 |
| TDLA30-300 | FR2 | 28 | 168 |
| TDLC300-100 | FR1 | 3.5 | 132 |
| TDLB100-400 | FR1 | 3.5 | 24 |

Source: 3GPP, 2020 and the author.

value. Consequently, they exhibit identical SER curves, and the simulations cover 5 distinct test conditions. The only difference between them lies in the inversion cycle limit (Section 3.4.2).

The test conditions are grouped into two mobility profiles based on UE speed (Table 2.1). The low-mobility profile includes TDLA30-5, TDLA30-10, and TDLA30-75, with speeds up to 3.1 km/h. The high-mobility profile comprises TDLA30-300, TDLC300-100, and TDLB100-400, with speeds ranging from 11.6 to 123.3 km/h. The ITU-R uses 3 km/h for indoor and pedestrian users, and 30 and 120 km/h for users in vehicles (ITU-R, 2017, Sec. 8.2 and Tab. 5). In each profile, the fastest condition is the most demanding: it has the smallest $T_c$ and the smallest $N$. This condition is TDLA30-75 in the low-mobility profile and TDLB100-400 in the high-mobility profile.

## 3.4 Inversion cycle limit

This section calculates, based on the coherence time, the maximum number of cycles between two inversions under each test condition.

### 3.4.1 Sample collection

The receiver first collects the $N$ noise samples for each PRB. Some of these samples arrive only after the data symbol (Section 2.2.1.6). Therefore, this work assumes that the receiver buffers the data symbols until the $N$ samples have been received. Subsequently, it processes the samples and equalizes the buffered data. A new set of samples arrives every interval $T_c$, and the processing must keep pace with this rate.

### 3.4.2 Throughput

The estimate $\hat{\mathbf{R}}_{cc}$ is unique per PRB within each interval $T_c$ (Section 2.2.1.6). The matrix $\mathbf{G}_c$ is inverted once and serves all iterations of the BCD-MMSE algorithm (Section 2.2.3). Thus, each DU performs one inversion of $\mathbf{G}_c$ per PRB every $T_c$. With $N_{\text{PRB}}$ PRBs, this results in $N_{\text{PRB}}$ inversions per DU every $T_c$. This work uses $N_{\text{PRB}} = 275$, which is the worst-case scenario for 5G NR (Section 2.1.2).

To track data collection, the module needs to complete the $N_{\text{PRB}}$ inversions within

$T_c$. Let $T_{\text{inv}}$ be the interval between the start of two inversions (Section 2.3.1.3). Then:

$$T_{\text{inv}} \leq \frac{T_c}{N_{\text{PRB}}}. \tag{3.1}$$

With a clock speed of 100 MHz (Section 3.2.1), this limit is given in cycles, rounded down. Table 3.4 shows the limit for each test condition.

Table 3.4: Maximum interval between inversions in each test condition.

| **Test condition** | $T_c$ **(ms)** | $T_c/N_{\text{PRB}}$ **(µs)** | **Limit at 100 MHz (cycles)** |
|---|---|---|---|
| TDLA30-5 | 25.0000 | 90.9091 | 9090 |
| TDLA30-10 | 12.5000 | 45.4545 | 4545 |
| TDLA30-75 | 1.6667 | 6.0606 | 606 |
| TDLA30-300 | 0.4167 | 1.5152 | 151 |
| TDLC300-100 | 1.2500 | 4.5455 | 454 |
| TDLB100-400 | 0.3125 | 1.1364 | 113 |

Source: the author. $T_c$ from Table 2.1; $N_{\text{PRB}} = 275$.

The hardware architectures in this work aim to meet this throughput limit under each test condition. The following are outside the scope: the delay between the arrival of a data symbol and its equalization; the throughput of the links between the DUs; and the other matrix inversions required by the BCD-MMSE algorithm, which the simulator performs using floating-point arithmetic (Section 3.1.3). The delay encompasses both the time spent waiting for samples and the latency of each inversion (i.e., the time elapsed from the start to the completion of an inversion).

## 3.5 Selection of the fixed-point format

### 3.5.1 SNR increase

To select the $\langle W, I \rangle$ format (Section 2.3.1.2), the BCD-MMSE algorithm was simulated using both fixed-point and 64-bit floating-point matrix inversion. Only the inversion of $\mathbf{G}_c$ differs between the two simulations; however, the SNR and SER are measured for the entire uplink. Each format was evaluated based on the SNR increase, that is, the additional SNR required by the uplink to achieve an $\text{SER} = 10^{-2}$ compared to the floating-point case. This increase corresponds to the horizontal distance between the two SER curves, which is the standard method for comparing two schemes (TSE; VISWANATH, 2005, p. 55).

Two thresholds were employed. A 0.1 dB threshold represents a near-ideal format, where the SER curve is virtually identical to that of the floating-point implementation. A 3 dB threshold serves as the maximum acceptable increase. An increase of 3 dB is equivalent to doubling the power (TSE; VISWANATH, 2005, p. 73). Consequently, at this threshold, each UE would need to transmit with twice the energy per symbol to compensate for the error introduced by the matrix inversion (Section 2.1.3).

### 3.5.2 Initial sweep

The first stage involved varying the integer and fractional parts separately, while keeping the other part fixed (Table 3.5). Each size was tested across 35 combinations of quantization and overflow modes (Table 2.5), using the BCD-MMSE algorithm with both 1 and 4 iterations.

Table 3.5: Sizes tested in the initial sweep.

| **Sweep** | **Integer bits** ($I$) | **Fractional bits** ($F$) |
|---|---|---|
| Integer part | 6, 8, 10, 12, 14, 16, 20, 24, 32 | 32 |
| Fractional part | 14 | 4, 6, 8, 10, 12, 16, 20, 24, 32 |

Source: the author.

### 3.5.3 Configurations of the initial sweep

The initial sweep utilized the 16 configurations for which the floating-point SER curve reaches $10^{-2}$ within the $-10$ to $10$ dB range. Running all of these across the 5 test conditions would have resulted in 80 cases, each involving 17 sizes and 35 mode combinations; therefore, only 5 configurations were selected.

The selection process relied on the $\mathbf{G}_c$ matrices at two SNR points surrounding the operating point (defined as $\mathrm{SER} = 10^{-2}$ in floating-point arithmetic). From each matrix, the largest absolute value between the real and imaginary parts of the elements was taken, since the circuit stores the two parts separately. This value is referred to here as the "maximum |part|". In each case, the 99th percentile of these values was used to ensure that the overall ranking would not be disproportionately influenced by a few outlier matrices. Larger elements necessitate a greater number of integer bits. Configurations 1G, 1H, 2E, and 2F contain the largest elements and are the most demanding, while configuration 1A

contains the smallest elements and was used as a baseline (Table 3.6).

These configurations also span the extremes of precision requirements. With a limited number of bits, the rounding error in matrix inversion increases with the condition number (OSINSKY et al., 2023, p. 1603). When the smallest eigenvalue is very small, the matrix approaches a singular state, making the result more dependent on the fractional part (Section 2.3.1.2). Configurations 2E and 2F exhibit the highest condition numbers (at the 99th percentile), whereas configuration 1A features the smallest eigenvalues (at the 1st percentile).

Table 3.6: $\mathbf{G}_c$ at the operating point of the initial-sweep configurations.

| | **SNR (dB)** | | **Largest \|part\|** | | **Condition number** | | **Smallest eigenvalue** | |
|---|---|---|---|---|---|---|---|---|
| **Configuration** | **192; 3.5** | **168; 28** | **192; 3.5** | **168; 28** | **192; 3.5** | **168; 28** | **192; 3.5** | **168; 28** |
| 1A | $-2.6$ | $-2.4$ | 63 | 66 | 57 | 58 | 1.9 | 1.9 |
| 1G | $-9.3$ | $-9.1$ | 2266 | 2383 | 187 | 169 | 22.4 | 24.1 |
| 1H | $-8.7$ | $-8.6$ | 2470 | 2385 | 195 | 172 | 22.1 | 24.7 |
| 2E | $-5.8$ | $-5.6$ | 962 | 999 | 286 | 254 | 8.8 | 9.7 |
| 2F | $-5.6$ | $-5.5$ | 983 | 987 | 282 | 285 | 8.8 | 9.5 |

Source: the author. Subcolumns: $N$; carrier in GHz.

### 3.5.4 Synthesis and selection of formats for validation

The sizes and modes of the initial sweep were also synthesized in Vitis HLS to measure resource occupancy (LUT, DSP, FF, and BRAM). The occupancy of each circuit is that of the most used resource in the FPGA. In each configuration and test condition, the integer and fractional parts with the lowest occupancy were chosen, with an SNR increase of up to 3 dB and of up to 0.1 dB, in 1 and in 4 iterations. Combining the two parts resulted in 10 formats (Table 3.7).

### 3.5.5 Validation

The initial sweep fixed one part of the word while varying the other, and used only 5 configurations. This can change the most used resource and the best mode. Therefore, the 10 formats were simulated again, with the 35 mode combinations, in the 24 configurations and 5 test conditions.

In this step, BCD-MMSE used only 1 iteration. The 4 iterations of the initial

Table 3.7: Configurations in which each format was chosen in the initial sweep.

| | 1 iteration | | 4 iterations | |
|---|---|---|---|---|
| **Format** | **Up to 3 dB** | **Up to 0.1 dB** | **Up to 3 dB** | **Up to 0.1 dB** |
| $\langle 14, 6\rangle$ | 1A | – | – | – |
| $\langle 16, 6\rangle$ | – | – | 1A | – |
| $\langle 18, 8\rangle$ | – | 1A | – | – |
| $\langle 20, 8\rangle$ | – | – | – | 1A |
| $\langle 22, 12\rangle$ | 1H, 2E, 2F | – | – | – |
| $\langle 24, 12\rangle$ | 1G | 2E, 2F | 1G, 1H, 2E, 2F | – |
| $\langle 26, 12\rangle$ | – | 1G[a], 1H[a] | – | 2E, 2F |
| $\langle 28, 12\rangle$ | – | – | – | 1G[a], 1H[a] |
| $\langle 28, 14\rangle$ | – | 1G[b], 1H[b] | – | – |
| $\langle 30, 14\rangle$ | – | – | – | 1G[b], 1H[b] |

Source: the author. Columns: SNR increase. No mark: all 5 test conditions. [a]Only TDLB100-400. [b]The other 4 conditions.

sweep had already helped to choose the formats of the most demanding cases: $\langle 16, 6\rangle$, $\langle 20, 8\rangle$, $\langle 28, 12\rangle$ and $\langle 30, 14\rangle$ only appeared with 4 iterations (Table 3.7). With 1 iteration, BCD-MMSE already has a good result in SER. Furthermore, the inverse of $\mathbf{G}_c$ is calculated once and used in all iterations, so the number of inversions does not change (Section 2.2.3).

The circuit has only one mode. Therefore, each format received a fixed mode in each group of configurations (Section 3.3). All configurations in a group use the same circuit. The mode was chosen according to the following criteria, in this order:

1. the mode that leaves more pairs (configuration, test condition) with an increase in SNR of up to 3 dB. A pair that does not reach $\mathrm{SER} = 10^{-2}$ counts as out of bounds;
2. the lowest resource occupancy, by the most used resource in the synthesis;
3. the quantization `AP_RND_ZERO`, which never uses more resources than the other tied modes;
4. the smallest increase in SNR in the worst pair;
5. the order of the modes in the list.

Then, with the fixed mode, three formats were chosen in each group. The first is the smallest format in which all configurations and conditions of the group reach $\mathrm{SER} = 10^{-2}$, with any increase in SNR. The second and third are the smallest, where all reach this point with an increase of up to 3 dB and up to 0.1 dB, respectively. These formats were implemented in Vivado and are in the Section 4.2.

## 3.6 Hardware architectures, test matrices and power

The chosen formats were synthesized using Vitis HLS. In all versions, the automatic loop pipelining feature was disabled (AMD, 2026, p. 42). The first version, without loop optimizations, was implemented for both $16 \times 16$ and $32 \times 32$ matrix sizes. In this version, the iterations of each loop execute sequentially, and the $N_{\text{PRB}} = 275$ matrices within a block are inverted one by one. Consequently, the interval between inversions exceeds the limits defined for all test conditions (Table 3.4), and the module does not track the arrival of the matrices at each $T_c$.

To meet the cycle limits for each test condition, a specific architecture was developed for every case, only in $16 \times 16$, with Vitis HLS directives that reduce the interval between inversions. All versions were implemented in Vivado, tested using $\mathbf{G}_c$ matrices obtained from the simulator (Section 3.6.2), and evaluated for power consumption and energy per inversion (Section 3.6.3). Each version produces the same inverse as the non-optimized implementation, a result verified bit-for-bit against the C-language code.

### 3.6.1 Hardware architectures

All optimized architectures employ pipelining (Section 2.3.1.3). Every $T_c$, the DU inverts 275 matrices $\mathbf{G}_c$, one for each PRB (Section 3.4). For the 9,090 and 4,545-cycle limits, the module inverts one matrix at a time. For tighter limits, it inverts the matrices in batches of $n$. In all cases, the module must receive the 275 matrices for a given block within the duration $T_c$.

The matrices within a batch are independent of one another. Consequently, $c$ of these matrices are computed in the same cycle using loop unrolling, which creates a copy of the loop body for each iteration. The loop that iterates over the $c$ matrices is nested within a pipelined loop, and the HLS tool performs the unrolling automatically (AMD, 2026, pp. 41, 372, and 428). When $c$ is less than $n$, groups of $c$ matrices are processed in alternating cycles. Each copy has its own dedicated memory bank, though it may share operators with the others.

The values for $n$ and $c$ were determined through trial and error and are presented in Table 3.8.

Table 3.8: Batch of the inversion module in each test condition.

| **Test condition** | **Limit per matrix (cycles)** | **Matrices per batch ($n$)** | **Copies ($c$)** | **Batches per block** |
|---|---|---|---|---|
| Without optimizations | – | 1 | 1 | 275 |
| TDLA30-5 | 9090 | 1 | 1 | 275 |
| TDLA30-10 | 4545 | 1 | 1 | 275 |
| TDLA30-75 | 606 | 12 | 6 | 23 |
| TDLC300-100 | 454 | 16 | 8 | 18 |
| TDLA30-300 | 151 | 32 | 2 | 9 |
| TDLB100-400 | 113 | 32 | 4 | 9 |

Source: the author. Limit per matrix from Table 3.4.

#### *3.6.1.1 Loop pipelining*

The module consists of six sequential stages (Figure 3.3): LU decomposition, extraction of the diagonal $\mathbf{D}$, calculation of the square root $\mathbf{D}^{1/2}$, computation of $\mathbf{F}$, forward substitution, and inversion. For the 9,090-cycle limit, the internal loops of these stages are annotated with the `#pragma HLS pipeline II=1` directive. The `D_extraction` stage is the only one without this directive, as it merely copies the diagonal from $\mathbf{U}$ to $\mathbf{D}$ and requires significantly fewer cycles than the other stages.

This directive enables loop pipelining, specifying that a new iteration should begin every cycle, that is, an initiation interval (II) of 1 cycle (Section 2.3.1.3). If data dependencies or resource constraints prevent this, the HLS tool will automatically use a larger II (KASTNER; MATAI; NEUENDORFFER, 2018, p. 45).

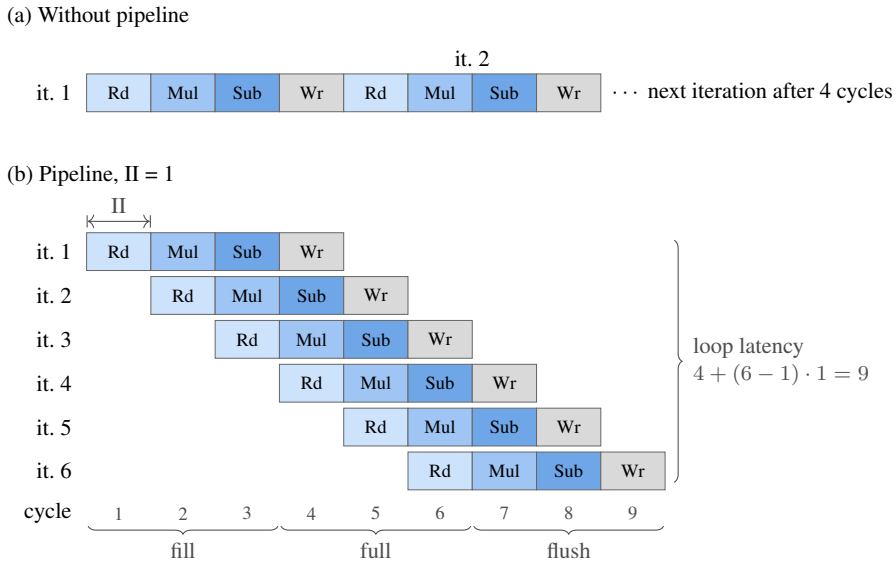
Figure 3.4 illustrates a loop from the LU decomposition process, both without and with pipelining. The individual stages and loops for a given matrix are executed sequentially. Consequently, the next inversion process cannot begin until the current one has finished; in this case, the interval between inversions is equal to the latency (KASTNER; MATAI; NEUENDORFFER, 2018, p. 114).

#### *3.6.1.2 Dataflow between stages*

The architecture of the 4,545-cycle limit maintains the loop pipeline and combines the six steps into four: LU decomposition, calculation of $\mathbf{F}$ (`D_extraction`, `D_square_root` and `G_decomposition`), forward substitution, and inversion. The `#pragma HLS dataflow` directive, applied to the top-level function, implements a task-level pipeline across the four stages, with each stage assigned to a specific matrix (Section 2.3.1.4). Each step carries the directive `#pragma HLS inline off`, which

Figure 3.4: Loop pipelining in the LU decomposition. The code is simplified.

```
for (line = column + 1; line < n; line++)
  for (k = column; k < n; k++) {
#pragma HLS pipeline II=1
    t = L[line][column] * U[column][k];
    U[line][k] = U[line][k] - t;
  }
```




Source: the author. Rd: read, Mul: multiply, Sub: subtract, Wr: write. Iteration latency of 4 cycles, for illustration.

keeps it as a separate function (KASTNER; MATAI; NEUENDORFFER, 2018, p. 52). In Figure 3.5, the latency of a matrix is the sum of the four steps, and the interval is the duration of the longest step.

### *3.6.1.3 Matrix batches*

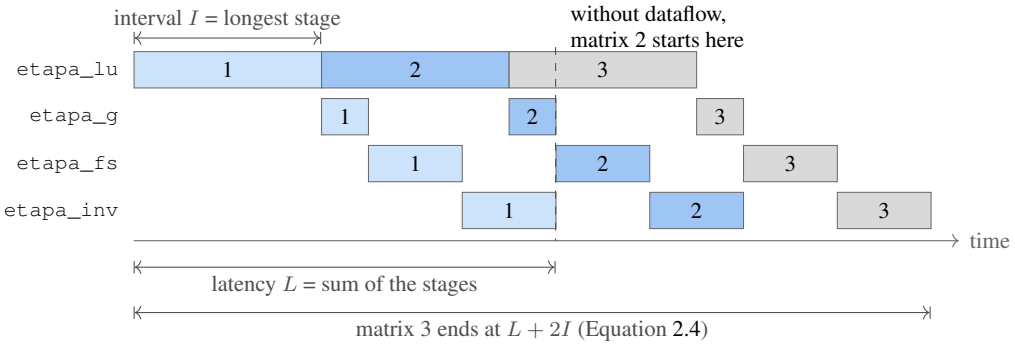
Within the 606 and 454 cycle limits, the steps form a dataflow, as in the 4,545 cycle limit. The $c$ copies read different matrices in the same cycle, but a BRAM memory only allows access to two addresses per cycle. Therefore, the batch memory is divided into $c$ BRAM banks, one per copy, using the directive `#pragma HLS array_partition cyclic` and factor $c$. Thus, matrix $m$ is located in bank $m \bmod c$ (KASTNER; MATAI; NEUENDORFFER, 2018, pp. 87–88).

Figure 3.6 shows the architecture of the 606 cycle limit. Since the two groups of 6 matrices alternate in each cycle, one iteration does not wait for the result of the previous one. Each copy has its own multipliers, but there is a single divider in each step, shared by the copies. It receives one matrix of the batch per cycle and, since it has a pipeline, it also does not wait for the previous division.

Figure 3.5: Dataflow between the stages of the inversion. The code is simplified.

```
void cholesky_inverse(A, inverse) {
#pragma HLS dataflow
  etapa_lu(A, L, U);
  etapa_g(U, L, G);
  etapa_fs(G, G_inv);
  etapa_inv(G_inv, inverse);
}

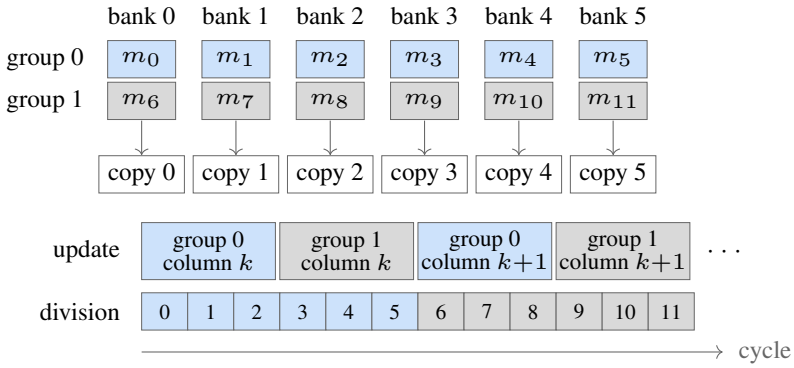
static void etapa_fs(G, G_inv) {
#pragma HLS inline off
  forward_substitution(G, G_inv);
}
```




Source: the author. Numbers in the boxes: index of the matrix.

Figure 3.6: Batch of 12 matrices with 6 copies, for the 606-cycle limit. The code is simplified.

```
#pragma HLS array_partition variable=U \
                        cyclic factor=COPIES dim=1
for (k = column; k < n; k++)
  for (g = 0; g < GRUPOS; g++) {
#pragma HLS pipeline II=1
    for (p = 0; p < COPIES; p++) {
      m = g * COPIES + p;
      t = L[m][line][column] * U[m][column][k];
      U[m][line][k] = U[m][line][k] - t;
    }
  }
```




Source: the author. Bank $p$ holds the matrices with $m \bmod 6 = p$. Update: 6 matrices per cycle. Division: one matrix $m$ per cycle, in a single divider.

*3.6.1.4 Systolic arrays*

Within the 151 and 113-cycle limits, the stages also form a dataflow, with each copy featuring its own divider. LU decomposition, forward substitution, and inversion utilize systolic arrays: the $k$-loop, which iterates over a row or a summation, is nested within a pipelined loop, which HLS unrolls into 16 processing elements (PEs) (Section 2.3.1.5). Each stage employs an array structure per copy, resulting in a total of $3c$ such arrays within the module.

In forward substitution and inversion, each PE adds a term and passes the partial sum to the next PE, following the same sequence as the unoptimized version, with each cycle computing a complete sum. In the LU decomposition, each cycle updates an entire row, with one element assigned to each PE. To achieve this, each row element is stored in its own dedicated register, configured using the `#pragma HLS array_partition complete` directive (KASTNER; MATAI; NEUENDORFFER, 2018, p. 87).

In both LU decomposition and forward substitution, each step reads rows that were written during the previous step, with a cycle offset corresponding to the step size. During the final steps of the LU decomposition and the initial steps of the forward substitution, which are relatively short, the required row might still be within the pipeline. Consequently, these steps are padded with empty cycles until they reach a minimum duration (Figure 3.7). Since HLS cannot automatically infer this from the source code, the `#pragma HLS dependence ... dependent=false` directive is used to specify that the iterations are independent of one another (KASTNER; MATAI; NEUENDORFFER, 2018, p. 149).

### 3.6.2 Test matrices

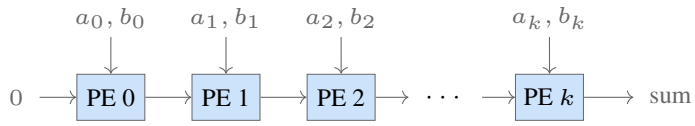
The hardware was tested with the same $\mathbf{G}_c$ matrices that BCD-MMSE inverts in the SER curves, with 1 iteration. These matrices were saved before inversion, in each of the 5 distinct test conditions (Section 3.3.2). In each condition, a sample was saved: some matrices randomly selected by configuration and SNR point, and the most extreme in range and precision, such as those with the largest element and the largest condition number. QPSK configurations were excluded because the modulation does not change $\mathbf{G}_c$.

The testbench inverts 20 matrices randomly selected from this sample. In the

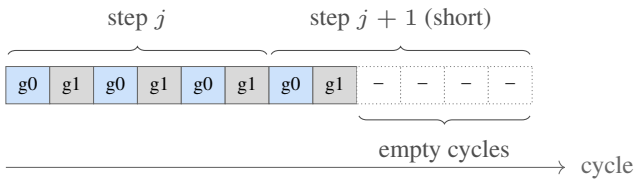
Figure 3.7: Systolic array of the forward substitution and minimum step of the LU decomposition. The code is simplified.

```
#pragma HLS array_partition variable=Ua \
                        complete dim=3
for (it = 0; it < LU_ITER; it++) {
#pragma HLS pipeline II=1
#pragma HLS dependence variable=Ua \
          type=inter dependent=false
  if (idx < reais)       // else: empty
    for (p = 0; p < COPIES; p++)
      for (k = 0; k < n; k++)  // PEs
        Ua[m][line][k] = row[k] - l*piv[k];
}
```

(a) PEs of one sum




(b) Steps of the LU decomposition




Source: the author. g0, g1: groups of matrices of the batch, one per cycle; there are 16 groups for the 151-cycle limit and 8 for the 113-cycle limit.

$16 \times 16$ circuit, these matrices come only from configurations 1A–1H. In architectures with batch, the 20 matrices are repeated until at least 3 batches are completed.

### 3.6.3 Power and energy

Power was estimated from the switching activity of the routed design: the version without optimizations under the 5 test conditions, and each optimized architecture only under its own. Vivado generates a functional netlist, which is simulated with the test-bench. The simulation checks the inverses byte by byte against the C code and records how many times each signal changes value in a SAIF (Switching Activity Interchange Format) file. Vivado then reads this file and reports the power.

Energy per inversion is defined as the dynamic power multiplied by the average time required for an inversion. This duration is calculated as the interval between two inversions, measured during the netlist simulation, multiplied by the 10 ns clock period. In architectures that employ batch processing, the interval between two batches is divided by $n$.

### 3.7 Organization of the results

The results are grouped according to the physical characteristics of the base station: the size of the inversion circuit ($16 \times 16$ or $32 \times 32$), the number of antennas $M$, and the number of clusters $C$. Within each group, the modulation, $K$, and IoTh vary. Thus, there are three groups: configurations 1A–1H ($M = 128$, $C = 8$, $16 \times 16$ circuit), 2A–2H ($M = 256$, $C = 8$, $32 \times 32$ circuit), and 3A–3H ($M = 256$, $C = 16$, $16 \times 16$ circuit).

# 4 RESULTADOS

## 4.1 Simulation setup

The simulation parameters are organized as the simulation flow of Figure 3.1. The base station configurations are inputs of both parts of the flow. The channels are generated before the simulation, by a MATLAB script that runs QuaDRiGa (Section 4.1.1). The Python simulator reads the stored channels and sets the other parameters: test conditions, signal and noise, receiver and evaluation (Section 4.1.2). In the tables, the source column gives the reference of each value, and the name in the code gives the variable that sets it. The parameter tables are in Appendix A.

### 4.1.1 Channel Generation in QuaDRiGa

Table A.1 shows the parameters of the MATLAB script that generates the channel matrices $\mathbf{H}$ and $\overline{\mathbf{H}}$. The remaining QuaDRiGa parameters were kept at their default values (JAECKEL; RASCHKOWSKI; THIELE, 2023).

The BS features a uniform linear array (ULA) of $M$ antennas, with a standard spacing of half a wavelength ($\lambda/2$) of the operating signal. The antennas at both the BS and the UEs are omnidirectional, meaning they provide the same gain in all directions.

For each realization, the script randomly determines the positions of the $K$ target UEs and the $r$ interferers. The distance is sampled such that the UE density remains uniform across the entire area. The random seed used for this sampling is always the realization number. Based on these positions, QuaDRiGa generates the frequency-domain channel response between each BS antenna and each UE within the propagation scenario.

In this work, the `3GPP_38.901_UMa_NLOS` propagation scenario was used, which represents an urban macrocell (UMa) with no line-of-sight (NLoS) between the base station (BS) and the user equipment (UEs). Table A.2 presents the valid range for the UMa propagation scenario and the values selected. The system model assumes a flat-fading channel, in which all subcarriers within a physical resource block (PRB) experience the same channel conditions.

### 4.1.2 Python Simulator

The Python simulator reads the stored channels and runs the rest of the simulation flow. Its parameters are grouped as in Figure 3.2: test conditions, signal and noise, receiver and evaluation. The test conditions are described in Section 3.3.2.

#### *4.1.2.1 Signal and Noise*

Table A.3 shows the parameters used to read the channels and to generate the transmitted symbols and the noise.

The SNR depends solely on the ratio between the symbol energy $E_s$ and the thermal noise power $\sigma^2$, as defined by the parameter `M5_Noise.snr_definition`. Accordingly, the constellations were normalized to have unit average energy (parameter `M4_Source.unit_energy_constellation`), and the symbol energy was set to $E_s = 1$ (parameter `M4_Source.Es`). Thus, the power $\sigma^2$ could be calculated based on the SNR to be simulated.

Furthermore, the power of the UEs was adjusted via channel normalization, defined by the parameter `M2_Channel.normalization`, ensuring that signals from UEs located at varying distances arrived at the BS with the same average power.

With the channel normalized, each of the $r$ interferers reaches every BS antenna with the same average power. In this scenario, the interference received at each antenna is scaled by the number of interferers, $r$. To ensure that the ratio of interference to thermal noise at each antenna equals the IoTh value specified in the configuration (parameter `M5_Noise.iot_db`), the power $\beta$ of each interferer is divided by $r$. This convention is governed by the parameter `M5_Noise.beta_convention` and follows the approach of averaging interference at each antenna, a method also employed in (TOMASIN et al., 2024).

The parameter `M5_Noise.data_noise_independent` specifies that the noise affecting the data symbols is drawn independently from the $N$ noise samples used to estimate the covariance matrix $\hat{\mathbf{R}}$. The parameter `M5_Noise.resample_noise_per_realization` specifies that the $N$ samples are re-drawn for each channel realization and each SNR point. This mimics the real behavior of the system, where samples come from the pilot REs and the data occupies other REs.

#### *4.1.2.2 Receiver*

Table A.4 shows the parameters for antenna partitioning, covariance estimation, and the equalizers.

Each cluster is assigned neighboring antennas (parameter `M3_Clustering.mapping`). The first cluster is assigned antennas 1 to $M_c$, the second is assigned antennas $M_c+1$ to $2M_c$, and so on. All clusters have the same number of antennas (parameter `M3_Clustering.equal_cluster_size`).

The equalizers receive the true $\mathbf{H}$ channel, without channel estimation (parameter `M7_Equalizers.perfect_csi`), as in (ZHAO et al., 2025). The BCD-MMSE was simulated with 1 iteration (parameter `M7_Equalizers.bcd_iterations`). The DUs update sequentially, from $c = 1$ to $C$, and each one uses the result of the previous one (parameter `M7_Equalizers.bcd_update_rule`) (ZHAO et al., 2025).

#### *4.1.2.3 Evaluation*

Table A.5 shows how the results are measured. The random number generator seed was set to 0, so that the results can be replicated.

## 4.2 Formato de ponto fixo

### 4.2.1 Mapas de aumento de SNR

O resultado da validação é mostrado em mapas de aumento de SNR, um para cada grupo de configurações. Cada linha do mapa é um formato, com o modo fixo do grupo. Cada coluna é uma configuração. A célula mostra o maior aumento de SNR entre as condições de teste. Os símbolos mostram as condições com aumento de SNR acima de 0,1 dB (vazados) e acima de 3 dB (cheios). Um x marca a configuração em que alguma condição não atinge $\mathrm{SER} = 10^{-2}$.

### 4.2.2 Formatos escolhidos

A Tabela 4.1 mostra os formatos escolhidos em cada grupo, pelo critério da Seção 3.5.

Table 4.1: Formatos escolhidos em cada grupo de configurações.

| Critério | Grupo 1 | Grupo 2 | Grupo 3 |
|---|---|---|---|
| Atinge SER $= 10^{-2}$ | $\langle 22, 12\rangle$ AP_RND_ZERO/AP_WRAP | $\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP | $\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP |
| Aumento de até 3 dB | $\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP | $\langle 28, 14\rangle$ AP_RND_ZERO/AP_WRAP | $\langle 24, 12\rangle$ AP_RND_ZERO/AP_WRAP |
| Aumento de até 0,1 dB | $\langle 28, 14\rangle$ AP_RND_ZERO/AP_WRAP | $\langle 30, 14\rangle$ AP_TRN/AP_SAT | $\langle 30, 14\rangle$ AP_TRN/AP_SAT |

Fonte: a autora. Abaixo do formato, o modo de quantização e o de estouro.

## 4.3 Vazão e atraso

As arquiteturas foram projetadas para atender o limite de vazão de cada condição de teste (Seção 3.4.2). Esta seção confere esse limite e mostra o atraso que resulta dessa escolha.

### 4.3.1 Vazão

A cada $T_c$, chega um bloco de $N_{\text{PRB}} = 275$ matrizes $\mathbf{G}_c$, uma por PRB. Nas condições TDLA30-75, TDLC300-100, TDLA30-300 e TDLB100-400, as matrizes são processadas em *batches* de 12, 16, 32 e 32 matrizes (Tabela 3.8). Um *batch* novo entra no módulo a cada intervalo de *batch*, antes de o anterior terminar. O tempo para o módulo aceitar todo o bloco é o número de *batches* vezes o intervalo de *batch*. Em todas as condições de teste e em todos os formatos, esse tempo fica abaixo de $T_c$. Assim, o módulo acompanha o ritmo de chegada das matrizes, e não se forma fila.

### 4.3.2 Atraso

O atraso vai da chegada da primeira amostra de um bloco até a última inversão desse bloco. Ele tem duas partes. A primeira é o tempo de coleta, que o receptor espera para juntar as $N$ amostras (Seção 3.4.1). Com $N$ amostras e 48 REs de DM-RS por PRB e por slot, a coleta leva $\lceil N/48 \rceil$ slots, limitada a $T_c$. A segunda é o tempo para inverter o bloco: a latência do primeiro *batch* mais o intervalo de cada *batch* seguinte. A Tabela 4.2 mostra essas duas partes no formato $\langle 24, 12\rangle$.

Table 4.2: Throughput and delay of one block of $N_{\mathrm{PRB}} = 275$ inversions, format $\langle 24, 12 \rangle$.

| Test condition | $T_c$ (ms) | Block input (ms) | Block end (ms) | Collection (ms) | Total delay (ms) |
|---|---|---|---|---|---|
| TDLA30-5 | 25.000 | 20.884 | 20.884 | 2.000 | 22.884 |
| TDLA30-10 | 12.500 | 8.305 | 8.354 | 2.000 | 10.354 |
| TDLA30-75 | 1.667 | 1.391 | 1.543 | 0.500 | 2.043 |
| TDLC300-100 | 1.250 | 1.159 | 1.336 | 1.250 | 2.586 |
| TDLA30-300 | 0.417 | 0.369 | 0.506 | 0.417 | 0.923 |
| TDLB100-400 | 0.313 | 0.184 | 0.265 | 0.313 | 0.578 |

Source: the author, from the netlist simulation at 100 MHz. Block input: time to accept the 275 matrices. Block end: time until the last inversion of the block. Total delay: collection plus block end.

Nas condições TDLC300-100 e TDLA30-300, a última inversão do bloco termina depois de $T_c$. Esse excesso não se acumula, pois o bloco seguinte já entra no módulo a tempo. Ele é um atraso fixo, de cerca de uma latência de *batch*, e não uma perda de vazão. Além disso, cada PRB só precisa da sua própria inversão. Os primeiros PRBs ficam prontos cerca de uma latência de *batch* depois da chegada do bloco. O atraso da tabela é o do último PRB.

A ITU-R prevê que o IMT-2020 possa oferecer latência de 1 ms pela interface aérea (ITU-R, 2015, p. 14). Essa latência é a contribuição da rede de rádio para o tempo entre o envio e a recepção de um pacote (ITU-R, 2015, p. 13). O processamento na BS é uma das parcelas da latência do plano de usuário (ITU-R, 2017, Sec. 7.2.6 e Tab. 3). Só nas condições TDLA30-300 e TDLB100-400 o atraso fica abaixo de 1 ms. Nas outras, o atraso vai de 2,0 a 22,9 ms.

Parte desse atraso vem do método de estimação, e não do hardware. Com $N = 192$ em FR1, a coleta leva 4 slots de 0,5 ms, ou seja, 2 ms. Esse tempo já passa de 1 ms antes de qualquer inversão. A outra parte vem da escolha de projeto. As arquiteturas das condições de baixa mobilidade distribuem as 275 inversões por quase todo o $T_c$. Com isso, elas gastam menos área e potência, mas o último PRB fica pronto mais tarde. Uma arquitetura mais rápida reduziria essa parte do atraso, com mais área e potência. A arquitetura da TDLB100-400, por exemplo, termina um bloco em 0,27 ms.

A coleta também depende do PUSCH. O DM-RS só é transmitido junto com o PUSCH, e o PUSCH só ocupa os slots em que os UEs alvo foram escalonados (Seção 2.1.2). Este trabalho supõe que todo slot leva PUSCH. As bandas n78 e n257 usam TDD (3GPP, 2020, Tabs. 5.2-1 e 5.2-2), e só os slots de *uplink* levam o PUSCH. Com um padrão TDD real, os slots de *uplink* ficam mais espaçados. A coleta das $N$

amostras leva mais tempo ou, se passar de $T_c$, o número de amostras disponível diminui.

# 5 CONCLUSIONS

Este trabalho estudou o BCD-MMSE no *uplink* de um sistema MIMO massivo com processamento descentralizado e ruído colorido. O foco foi a inversão da matriz $\mathbf{G}_c$ em cada DU, feita em ponto fixo em um FPGA. Um simulador em Python mediu o efeito do formato de ponto fixo nas curvas de SER. Depois, o módulo de inversão foi implementado em hardware, com uma arquitetura para cada condição de teste.

[completar] Formatos escolhidos para cada grupo de configurações e aumento de SNR de cada um, em relação ao ponto flutuante (Seção 4.2).

[completar] Recursos e potência das arquiteturas. Todas atendem o limite de vazão da sua condição de teste.

As arquiteturas foram dimensionadas pela vazão, e não pelo atraso. Nas condições de baixa mobilidade, elas distribuem as inversões ao longo do tempo de coerência e gastam menos área e potência. Em troca, o atraso até a última inversão de um bloco fica maior. Parte do atraso vem do próprio método de estimação, pois juntar as $N$ amostras de ruído leva tempo.

## 5.1 Future work

Um trabalho futuro pode incluir um prazo de atraso no dimensionamento do hardware e medir quanto ele custa em área e potência. Também pode considerar um padrão TDD real, que muda o tempo de coleta e o número de amostras disponível. Outras extensões são levar ao hardware as outras inversões do BCD-MMSE, que hoje ficam em ponto flutuante, e avaliar mais de uma iteração do algoritmo.

## REFERENCES


3GPP. *NR; Base Station (BS) radio transmission and reception*. [S.l.], 2020.

3GPP. *NR; Physical channels and modulation*. [S.l.], 2025.

AMD. **Vitis High-Level Synthesis User Guide (UG1399)**. 2026.1. ed. [S.l.], 2026. Disponível em: docs.amd.com.

ITU-R. **IMT Vision – Framework and overall objectives of the future development of IMT for 2020 and beyond**. [S.l.], 2015.

ITU-R. **Guidelines for evaluation of radio interface technologies for IMT-2020**. [S.l.], 2017.

JAECKEL, S.; RASCHKOWSKI, L.; THIELE, L. **QuaDRiGa – Quasi Deterministic Radio Channel Generator: User Manual and Documentation**. Document revision v2.8.1. Berlin, 2023.

JEON, C. et al. Decentralized equalization with feedforward architectures for massive mu-mimo. **IEEE Transactions on Signal Processing**, v. 67, n. 17, p. 4418–4432, 2019.

KASTNER, R.; MATAI, J.; NEUENDORFFER, S. **Parallel Programming for FPGAs**. [S.l.]: arXiv, 2018. ArXiv:1805.03648.

LEDOIT, O.; WOLF, M. A well-conditioned estimator for large-dimensional covariance matrices. **Journal of Multivariate Analysis**, v. 88, n. 2, p. 365–411, 2004.

LIN, X.; LEE, N. (Ed.). **5G and Beyond: Fundamentals and Standards**. Cham: Springer, 2021.

OSINSKY, A. et al. Regularization for Cholesky decomposition in massive MIMO detection. **IEEE Wireless Communications Letters**, v. 12, n. 9, p. 1603–1607, 2023.

TOMASIN, S. et al. Estimation of interference correlation in mmwave cellular systems. **IEEE Transactions on Wireless Communications**, v. 23, n. 3, p. 1771–1782, 2024.

TSE, D.; VISWANATH, P. **Fundamentals of Wireless Communication**. Cambridge: Cambridge University Press, 2005.

WANG, H. et al. An efficient detector for massive mimo based on improved matrix partition. **IEEE Transactions on Signal Processing**, v. 69, p. 2971–2986, 2021.

ZHAO, X. et al. Decentralized linear mmse equalizer under colored noise for massive mimo systems. In: **2021 IEEE Global Communications Conference (GLOBECOM)**. [S.l.: s.n.], 2021.

ZHAO, X. et al. Efficient lmmse equalization for massive mimo systems under decentralized baseband processing architecture. **IEEE Journal on Selected Areas in Communications**, v. 43, n. 3, p. 736–751, 2025.

ZHUANG, Z. et al. Decentralized mimo systems with imperfect csi using lmmse receivers. **IEEE Journal of Selected Topics in Signal Processing**, v. 19, n. 2, p. 331–348, 2025.

# APPENDIX A — SIMULATION PARAMETERS

Table A.1: Channel parameters set in the MATLAB script that runs QuaDRiGa.

| Source | Parameter | Name in the MATLAB script | Value |
|---|---|---|---|
| ZHAO et al., 2025 | Number of cells and BSs | `tx_position` (one BS) | 1 |
| | Sector where the UEs are | `sector_deg` | 120° |
| | Position of the interferers | `n_ue = K + r` | Same rule as the served UEs |
| | Channel generator | – | QuaDRiGa |
| | Channel constant within a PRB | – | Flat channel |
| QuaDRiGa default JAECKEL; RASCHKOWSKI; THIELE, 2023 | Uses only the minimal version of the 3GPP model | `use_3GPP_baseline` | `false` |
| | Shape of the spatial correlation of the large-scale parameters | `autocorrelation_function` | `Comb300` |
| | Adds the direct-path delay to the delays | `use_absolute_delays` | `false` |
| | Each path starts with a random phase | `use_random_initial_phase` | `true` |
| | Samples per half wavelength along the track | `sample_density` | 2.5 |
| Defined in this work | QuaDRiGa version | `version` | 2.8.1-0 |
| | Propagation scenario | `scenario` | `3GPP_38.901_UMa_NLOS` (Table A.2) |
| | Bandwidth | `bandwidth` | 20 MHz |
| | Flat-channel extraction | `fr(bandwidth, 1)` | Response at one subcarrier |
| | BS array | `tx_array` | ULA with $\lambda/2$ spacing |
| | Antenna elements (BS and UE) | `qd_arrayant('omni')` | Omnidirectional |
| | Sector orientation | `tx_array.ang` | Centered on the direction perpendicular to the array |
| | UE placement | `rx_track` | Uniform over the sector area |
| | Realizations | `rng(seed + rr)` | New positions in each realization |

Source: ZHAO et al., 2025; JAECKEL; RASCHKOWSKI; THIELE, 2023 and the author.

Table A.2: Parameters of the propagation scenario `3GPP_38.901_UMa_NLOS`: valid range and values used in the simulation.

| Set in | Parameter | Name | Valid range of the propagation scenario | Value used |
|---|---|---|---|---|
| MATLAB script | Carrier frequency | `center_freq` | 0.5 to 100 GHz | 3.5 GHz (FR1) and 28 GHz (FR2) |
| | BS–UE distance | `r_min_m`, `r_max_m` | 10 to 5000 m | 50 to 100 m |
| | BS height | `bs_height_m` | 25 m | 25 m |
| | UE height | `ue_height_m` | 1.5 to 22.5 m | 1.5 m |
| Scenario file (`.conf`) | Number of clusters | `NumClusters` | 21 | |
| | Paths per cluster | `NumSubPaths` | 20 | |
| | Shadowing standard deviation | `SF_sigma` | 6 dB | |

Source: JAECKEL; RASCHKOWSKI; THIELE, 2023; ZHAO et al., 2025 and the author.

Table A.3: Signal and noise parameters.

| Source | Parameter | Name in the Python simulator | Value |
|---|---|---|---|
| ZHAO et al., 2025 | Noise model | `M5_Noise.noise_model` | $\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \beta\overline{\mathbf{H}}\,\overline{\mathbf{H}}^H + \sigma^2\mathbf{I})$ |
| | SNR definition | `M5_Noise.snr_definition` | $10\log_{10}(E_s/\sigma^2)$ |
| | Mean symbol energy | `M4_Source.Es` | $E_s = 1$ |
| | Thermal noise power | `M5_Noise.snr_definition` | $\sigma^2 = 10^{-\mathrm{SNR}/10}$ |
| Defined in this work | Channel input | `M2_Channel.backend` | $\mathbf{H}$ and $\overline{\mathbf{H}}$ read from the `.npz` file of QuaDRiGa |
| | Channel normalization | `M2_Channel.normalization` | Same mean power for every UE |
| | Constellation energy | `M4_Source.unit_energy_constellation` | Unit mean |
| | Interferer power | `M5_Noise.beta_convention` | $\beta = \sigma^2(10^{\mathrm{IoTh}/10} - 1)/r$ |
| | Noise of the data symbols | `M5_Noise.data_noise_independent` | Independent of the $N$ samples |

Source: ZHAO et al., 2025 and the author. IoTh and modulation are in Table 3.2.

Table A.4: Receiver parameters: partitioning, covariance estimation and equalization.

| Source | Parameter | Name in the Python simulator | Value |
|---|---|---|---|
| ZHAO et al., 2025 | Antennas per cluster | `M3_Clustering.equal_cluster_size` | $M_c = M/C$ |
| | Noise samples to estimate $\hat{\mathbf{R}}$ | `M5_Noise.n_noise_samples` | $N = 192$ |
| | Covariance estimate | `M6_Covariance.estimator` | Sample mean of the $N$ samples |
| | Channel knowledge at the BS | `M7_Equalizers.perfect_csi` | Perfect $\mathbf{H}$ |
| | Compared equalizers | `M7_Equalizers.curves` | LMMSE, ZF and BCD-MMSE |
| | BCD-MMSE iterations | `M7_Equalizers.bcd_iterations` | 1 |
| | BCD update order | `M7_Equalizers.bcd_update_rule` | Sequential, $c = 1, \ldots, C$ |
| | Computation of the equalizer $\mathbf{W}$ | `M1_Driver.recompute_W_per_realization` | Once per realization and SNR point |
| Defined in this work | Antenna partitioning into clusters | `M3_Clustering.mapping` | Contiguous |
| | $N$ in each test condition | `M1_Driver.test_conditions` | 192, 168, 132 or 24 (Table 3.3) |
| | Noise sample draws | `M5_Noise.resample_noise_per_realization` | New samples at each realization and SNR point |
| | Conditioning of $\hat{\mathbf{R}}$ | `M6_Covariance.shrinkage` | Ledoit–Wolf shrinkage |
| | ZF form | `M7_Equalizers.zf_form` | $(\mathbf{H}^H\mathbf{H})^{-1}\mathbf{H}^H$ |

Source: ZHAO et al., 2025 and the author. $C$ and $K$ are in Table 3.2. Names in the code: `dbp_sim/params.py`.

Table A.5: Evaluation parameters.

| Source | Parameter | Name in the Python simulator | Value |
|---|---|---|---|
| ZHAO et al., 2025 | Channel realizations | `M1_Driver.n_realizations` | 100 |
| | Symbols per realization | `M1_Driver.n_symbols` | 480 |
| | Detection | `M8_Metric.detection` | Nearest constellation point |
| | Metric | `M8_Metric.metric` | Symbol error rate (SER) |
| | SNR range | `M1_Driver.snr_db_grid` | $-10$ to 10 dB, 2 dB step |
| Defined in this work | Bias removal before detection | `M8_Metric.bias_removal` | No |
| | Random seed | `M1_Driver.rng_seed` | 0 |

Source: ZHAO et al., 2025 and the author. Names in the code: `dbp_sim/params.py`.